

Amazon_Product_Recommendation_System

March 15, 2026

```
[4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[5]: !gdown 1_A0qhk2RddHKHE-3nLJrDhlFUWdmtuUA
```

Downloading...

From: https://drive.google.com/uc?id=1_A0qhk2RddHKHE-3nLJrDhlFUWdmtuUA

To: /content/amazon.csv

0% 0.00/4.74M [00:00<?, ?B/s] 55% 2.62M/4.74M [00:00<00:00, 21.7MB/s] 100%
4.74M/4.74M [00:00<00:00, 21.5MB/s]

```
[6]: amazon = pd.read_csv('/content/amazon.csv')

amazon.head()
```

```
[6]:      product_id      product_name \
0  B07JW9H4J1  Wayona Nylon Braided USB to Lightning Fast Cha...
1  B098NS6PVG  Ambrane Unbreakable 60W / 3A Fast Charging 1.5...
2  B096MSW6CT  Sounce Fast Phone Charging Cable & Data Sync U...
3  B08HJDJ86NZ  boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...
4  B08CF3B7N1  Portronics Konnect L 1.2M Fast Charging 3A 8 P...

      category discounted_price \
0  Computers&Accessories|Accessories&Peripherals|...      399
1  Computers&Accessories|Accessories&Peripherals|...      199
2  Computers&Accessories|Accessories&Peripherals|...      199
3  Computers&Accessories|Accessories&Peripherals|...      329
4  Computers&Accessories|Accessories&Peripherals|...      154

      actual_price discount_percentage rating rating_count \
0           1,099           64%      4.2      24,269
1           349           43%      4.0      43,994
2          1,899           90%      3.9       7,928
3           699           53%      4.2      94,363
4           399           61%      4.2      16,905
```

about_product \

0 High Compatibility : Compatible With iPhone 12...

1 Compatible with all Type C enabled devices, be...

2 Fast Charger& Data Sync -With built-in safet...

3 The boAt Deuce USB 300 2 in 1 cable is compati...

4 [CHARGE & SYNC FUNCTION]- This cable comes wit...

user_id \

0 AG3D604STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB...

1 AECPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX...

2 AGU3BBQ2V2DDAMOAKGFAWDDQ6QHA,AESFLDV2PT363T2AQ...

3 AEWAZDZZJLQUYVOVGBEUKSLXHQ5A,AG5HTSFRRE6NL3M5S...

4 AE3Q6KSUK5P75D5HFYHCRAOLODSA,AFUGIFH5ZAFXRDSZH...

user_name \

0 Manav,Adarsh gupta,Sundeep,S.Sayeed Ahmed,jasp...

1 ArdKn,Nirbhay kumar,Sagar Viswanathan,Asp,Plac...

2 Kunal,Himanshu,viswanath,sai niharka,saqib mal...

3 Omkar dhale,JD,HEMALATHA,Ajwadh a.,amar singh ...

4 rahuls6099,Swasat Borah,Ajay Wadke,Pranali,RVK...

review_id \

0 R3HXWTOLRPONMF,R2AJM3LFTLZHF0,R6AQJGUP6P86,R1K...

1 RGIQEG07R9HS2,R1SMWZQ86XIN8U,R2J3Y1WL29GWDE,RY...

2 R3J3EEQQ9TZI5ZJ,R3E7WBGK7ID0KV,RWU79XKQ6I1QF,R2...

3 R3EEUZKKK9J36I,R3HJVYCLYOY554,REDECAZ7AMPQC,R1...

4 R1BP4L2HH9TFUP,R16PVJEXKV6QZS,R2UPDB81N66T4P,R...

review_title \

0 Satisfied,Charging is really fast,Value for mo...

1 A Good Braided Cable for Your Type C Device,Go...

2 Good speed for earlier versions,Good Product,W...

3 Good product,Good one,Nice,Really nice product...

4 As good as original,Decent,Good one for second...

review_content \

0 Looks durable Charging is fine tooNo complains...

1 I ordered this cable to connect my phone to An...

2 Not quite durable and sturdy,https://m.media-a...

3 Good product,long wire,Charges good,Nice,I bou...

4 Bought this instead of original apple, does th...

img_link \

0 https://m.media-amazon.com/images/W/WEBP_40237...

1 https://m.media-amazon.com/images/W/WEBP_40237...

2 https://m.media-amazon.com/images/W/WEBP_40237...

3 https://m.media-amazon.com/images/I/41V5FtEWPk...

4 https://m.media-amazon.com/images/W/WEBP_40237...

```
                                product_link
0  https://www.amazon.in/Wayona-Braided-WN3LG1-Sy...
1  https://www.amazon.in/Ambrane-Unbreakable-Char...
2  https://www.amazon.in/Sounce-iPhone-Charging-C...
3  https://www.amazon.in/Deuce-300-Resistant-Tang...
4  https://www.amazon.in/Portronics-Konnect-POR-1...
```

```
[7]: amazon.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1465 entries, 0 to 1464
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            1465 non-null   object
1   product_name          1465 non-null   object
2   category              1465 non-null   object
3   discounted_price      1465 non-null   object
4   actual_price          1465 non-null   object
5   discount_percentage   1465 non-null   object
6   rating                1465 non-null   object
7   rating_count          1463 non-null   object
8   about_product         1465 non-null   object
9   user_id               1465 non-null   object
10  user_name             1465 non-null   object
11  review_id             1465 non-null   object
12  review_title          1465 non-null   object
13  review_content        1465 non-null   object
14  img_link              1465 non-null   object
15  product_link          1465 non-null   object
dtypes: object(16)
memory usage: 183.3+ KB
```

0.1 Section A: Data Understanding & Cleaning

Number of unique users, products, and reviews

```
[8]: print(f"Total number of users: {amazon['user_id'].count()}")
      print(f"Total number of unique users: {amazon['user_id'].nunique()}")
      print('\n')

      print(f"Total number of products: {amazon['product_id'].count()}")
      print(f"Total number of unique products: {amazon['product_id'].nunique()}")
      print('\n')

      print(f"Total number of reviews: {amazon['review_id'].count()}")
```

Total number of users: 1465
Total number of unique users: 1194

Total number of products: 1465
Total number of unique products: 1351

Total number of reviews: 1465

Top 5 categories by number of products

```
[9]: # Top 5 categories with most number of products
print('Top 5 categories with most number of products')
print('_' * 45)
amazon.groupby('category')['product_id'].nunique().sort_values(ascending=False).
    ↪reset_index(name='Total Products').head(5)
```

Top 5 categories with most number of products

```
[9]:
```

	category	Total Products
0	Computers&Accessories Accessories&Peripherals ...	161
1	Electronics Mobiles&Accessories Smartphones&Ba...	68
2	Electronics WearableTechnology SmartWatches	62
3	Electronics HomeTheater,TV&Video Televisions S...	60
4	Electronics Headphones,Earbuds&Accessories Hea...	51

Cleanups

```
[10]: # Count how many of each Python type exist in the 'rating' column
print('Rating type before cleanup')
rating_type_counts = amazon['rating'].apply(type).value_counts()
print(rating_type_counts)
display(amazon['rating'].head(5))
print('\n')

# Convert rating values to float
amazon['rating'] = pd.to_numeric(amazon['rating'], errors='coerce')
amazon['rating'] = amazon['rating'].astype(float)

print('Rating type after cleanup')
rating_type_counts = amazon['rating'].apply(type).value_counts()
print(rating_type_counts)
display(amazon['rating'].head(5))
```

Rating type before cleanup
rating
<class 'str'> 1465
Name: count, dtype: int64

```
0    4.2
1    4.0
2    3.9
3    4.2
4    4.2
Name: rating, dtype: object
```

```
Rating type after cleanup
rating
<class 'float'>    1465
Name: count, dtype: int64
```

```
0    4.2
1    4.0
2    3.9
3    4.2
4    4.2
Name: rating, dtype: float64
```

```
[11]: # Get min and max ratings
min_rating = amazon['rating'].min()
max_rating = amazon['rating'].max()

print(f"Min rating: {min_rating}")
print(f"Max rating: {max_rating}")
```

```
Min rating: 2.0
Max rating: 5.0
```

```
[12]: # Count how many of each Python type exist in the 'rating_count' column
print('Rating count type before cleanup')
rating_count_type_counts = amazon['rating_count'].apply(type).value_counts()
print(rating_count_type_counts)
display(amazon['rating_count'].head(5))
print('\n')
```

```
Rating count type before cleanup
rating_count
<class 'str'>    1463
<class 'float'>    2
Name: count, dtype: int64

0    24,269
1    43,994
2     7,928
3    94,363
4    16,905
Name: rating_count, dtype: object
```

Looks like there are a couple of existing float values in the rating count column, let's check that so we know how to handle them.

```
[13]: # Identify and print rows where the value is already a float
numeric_mask = amazon['rating_count'].apply(lambda x: isinstance(x, (float)))
numeric_rows = amazon[numeric_mask]

print("Rows that are already numeric:")
print(numeric_rows[['product_name', 'rating_count']])
```

Rows that are already numeric:

	product_name	rating_count
282	Amazon Brand - Solimo 65W Fast Charging Braide...	NaN
324	REDTECH USB-C to Lightning Cable 3.3FT, [Apple...	NaN

Just a couple of NaN values, which we can easily fill up with 0.

```
[14]: # Convert 'rating_count' values to float
amazon['rating_count'] = amazon['rating_count'].str.replace(',', '')
amazon['rating_count'] = pd.to_numeric(amazon['rating_count'], errors='coerce')
amazon['rating_count'] = amazon['rating_count'].fillna(0).astype(int)

print('Rating count type after cleanup')
rating_count_type_counts = amazon['rating'].apply(type).value_counts()
display(amazon['rating_count'].head(5))
print(rating_count_type_counts)
print(f"\n rating count null count: {amazon['rating_count'].isna().sum()}")
```

Rating count type after cleanup

```
0    24269
1    43994
2     7928
3    94363
4    16905
Name: rating_count, dtype: int64
```

```
rating
<class 'float'>    1465
Name: count, dtype: int64
```

rating count null count: 0

```
[15]: # Count how many of each Python type exist in the 'actual_price' column before_
      ↪ cleanup
print('Actual price type before cleanup - with the currency symbol')
print('_' * 60)
actual_price_type_counts = amazon['actual_price'].apply(type).value_counts()
```

```

print(actual_price_type_counts)
display(amazon['actual_price'].head(5))
print('\n')

# Count how many of each Python type exist in the 'discounted_price' column
↳ before cleanup
print('Discounted price type before cleanup - with the currency symbol')
print('_' * 65)
discounted_price_type_counts = amazon['discounted_price'].apply(type).
↳ value_counts()
print(discounted_price_type_counts)
display(amazon['discounted_price'].head(5))
print('\n')

# Convert to float after removing the currency symbol
amazon['actual_price'] = amazon['actual_price'].str.replace('[^0-9.]', '',
↳ regex=True).astype(float)

# Count how many of each Python type exist in the 'actual_price' column after
↳ cleanup
print('Actual price type after cleanup')
print('_' * 30)
actual_price_type_counts = amazon['actual_price'].apply(type).value_counts()
print(actual_price_type_counts)
display(amazon['actual_price'].head(5))
print('\n')

# Convert to float after removing the currency symbol
amazon['discounted_price'] = amazon['discounted_price'].str.replace('[^0-9.]',
↳ '', regex=True).astype(float)

# Count how many of each Python type exist in the 'discounted_price' column
↳ after cleanup
print('Discounted price type after cleanup')
print('_' * 35)
discounted_price_type_counts = amazon['discounted_price'].apply(type).
↳ value_counts()
print(discounted_price_type_counts)
display(amazon['discounted_price'].head(5))

```

Actual price type before cleanup - with the currency symbol

```

-----
actual_price
<class 'str'>    1465
Name: count, dtype: int64

0    1,099
1     349

```

```
2    1,899
3      699
4      399
Name: actual_price, dtype: object
```

Discounted price type before cleanup - with the currency symbol

```
-----
discounted_price
<class 'str'>      1465
Name: count, dtype: int64

0      399
1      199
2      199
3      329
4      154
Name: discounted_price, dtype: object
```

Actual price type after cleanup

```
-----
actual_price
<class 'float'>    1465
Name: count, dtype: int64

0    1099.0
1     349.0
2    1899.0
3     699.0
4     399.0
Name: actual_price, dtype: float64
```

Discounted price type after cleanup

```
-----
discounted_price
<class 'float'>    1465
Name: count, dtype: int64

0     399.0
1     199.0
2     199.0
3     329.0
4     154.0
Name: discounted_price, dtype: float64
```



```
[16]: # Count how many of each Python type exist in the 'discount_percentage' column
      ↪ before cleanup
print('Discount percentage before cleanup - with the currency symbol')
print('_' * 60)
discount_percentage_type_counts = amazon['discount_percentage'].apply(type).
      ↪ value_counts()
display(amazon['discount_percentage'].head(5))
print(discount_percentage_type_counts)
print('\n')

# Strip out the % symbol and convert the number string to float
amazon['discount_percentage'] = amazon['discount_percentage'].str.replace('%',
      ↪ '').astype(float)

# Count how many of each Python type exist in the 'discounted_percentage'
      ↪ column after cleanup
print('Discount percentage type after cleanup')
print('_' * 40)
discount_percentage_type_counts = amazon['discount_percentage'].apply(type).
      ↪ value_counts()
display(amazon['discount_percentage'].head(5))
print(discount_percentage_type_counts)
```

Discount percentage before cleanup - with the currency symbol

```
-----
0    64%
1    43%
2    90%
3    53%
4    61%
Name: discount_percentage, dtype: object

discount_percentage
<class 'str'>    1465
Name: count, dtype: int64
```

Discount percentage type after cleanup

```
-----
0    64.0
1    43.0
2    90.0
3    53.0
4    61.0
Name: discount_percentage, dtype: float64

discount_percentage
```

```
<class 'float'>    1465
Name: count, dtype: int64
```

How many interactions exist per user and per product at a high level. This gives an early sense of whether most users interact with many products or only a few

```
[17]: reviews_per_user = amazon.groupby('user_id')['review_id'].count().
      ↪sort_values(ascending=False).reset_index(name='Total Reviews')

      display(reviews_per_user.head(10))

      total_users = len(reviews_per_user)
      users_review_count_1_plus = len(reviews_per_user[reviews_per_user['Total_
      ↪Reviews'] > 1])
      users_review_count_2_plus = len(reviews_per_user[reviews_per_user['Total_
      ↪Reviews'] > 2])

      print("-" * 70)
      print(f"Number of users who have rated more than 1 product:␣
      ↪{users_review_count_1_plus}\n")
      print(f"Number of users who have rated more than 2 products:␣
      ↪{users_review_count_2_plus}\n")
      print(f"Percentage of users who have rated more than 1 product␣
      ↪{round((users_review_count_1_plus / total_users) * 100, 2)}%")
```

	user_id	Total Reviews
0	AHIKJUDTVJ4T6DV6IUGFYZ5LXMPA,AE55KTFVNXFYFD5FPY...	10
1	AG3D604STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB...	8
2	AGAELRYPMTG5SADZPDYB343EASAA,AGFN4JODOM2NTFCJQ...	8
3	AEWAZDZZJLQUYVOVGBEUKSLXHQA,AG5HTSFRRE6NL3M5S...	7
4	AECPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX...	7
5	AHWRZWPCTG6ICA7WTNLNNZXWFI5Q,AF2AASVYVSROFD7FX...	6
6	AG44HJB2AMIVHAGQZ2WGWONERKCA,AHL2FABQV6XAHZN54...	6
7	AHEVOQADJSSRX7DS325HSFLMP7VQ,AG7XYZRCSKX6G2OLO...	5
8	AEC6UDCEAUIBIFHGQDQ4KR67GC4A,AHRKSUOZXKKDERRY3...	5
9	AGU76WKSU62DUNTPCMT4FCUNRTQ,AEOVR6JEQTAC77BXE...	5

Number of users who have rated more than 1 product: 144

Number of users who have rated more than 2 products: 69

Percentage of users who have rated more than 1 product 12.06%

```
[18]: reviews_per_product = amazon.groupby('product_id')['review_id'].count().
      ↪sort_values(ascending=False).reset_index(name='Total Reviews')

      display(reviews_per_product.head(10))
```

```

total_products = len(reviews_per_product)
product_reviews_count_1_plus =
    ↳ len(reviews_per_product[reviews_per_product['Total Reviews'] > 1])
product_reviews_count_2_plus =
    ↳ len(reviews_per_product[reviews_per_product['Total Reviews'] > 2])

print("-" * 70)
print(f"Number of products with more than 1 review:
    ↳ {product_reviews_count_1_plus}\n")
print(f"Number of products with more than 2 reviews:
    ↳ {product_reviews_count_2_plus}\n")
print(f"Percentage of products with more than 1 review
    ↳ {round((product_reviews_count_1_plus / total_products) * 100, 2)}%")

```

	product_id	Total Reviews
0	B09W5XR9RT	3
1	B08R69VDHT	3
2	B09NHVCHS9	3
3	B082T6V3DT	3
4	B085DTN6R2	3
5	B09YLXYP7Y	3
6	B083342NKJ	3
7	B07XLCFSSN	3
8	B09CMP1SC8	3
9	B098NS6PVG	3

Number of products with more than 1 review: 92

Number of products with more than 2 reviews: 22

Percentage of products with more than 1 review 6.81%

```

[19]: product_rating_count = amazon.groupby('product_id')['rating_count'].sum().
    ↳ reset_index(name='Total Ratings')

# sort_values(ascending=False).reset_index()

# Top 10 products with highest rating count
print("Top 10 products with highest rating count")
print("-" * 40)
display(product_rating_count.sort_values(by='Total Ratings', ascending=False).
    ↳ head(10))

total_products = len(product_rating_count)
product_rating_count_100_plus =
    ↳ len(product_rating_count[product_rating_count['Total Ratings'] > 100])

```

```

product_rating_count_500_plus =
    ↳ len(product_rating_count[product_rating_count['Total Ratings'] > 500])
product_rating_count_1000_plus =
    ↳ len(product_rating_count[product_rating_count['Total Ratings'] > 1000])

print("-" * 70)
print(f"Number of products with more than 100 rating counts:
    ↳ {product_rating_count_100_plus}\n")
print(f"Number of products with more than 500 rating counts:
    ↳ {product_rating_count_500_plus}\n")
print(f"Number of products with more than 1000 rating counts:
    ↳ {product_rating_count_1000_plus}\n")
print(f"Percentage of products with more than 100 rating counts
    ↳ {round((product_rating_count_100_plus / total_products) * 100, 2)}%")

```

Top 10 products with highest rating count

```

-----
      product_id  Total Ratings
356  B07KSMBL2H          853945
138  B014I8SX4Y          426973
137  B014I8SSD0          426973
160  B01DEWVZ2C          385177
318  B07GQD4K6L          363713
317  B07GPXXNNG          363713
232  B071Z8M4KX          363711
907  B09GFLXVH9          313836
911  B09GFPVD9Y          313836
910  B09GFPN6TP          313832

```

```

-----
Number of products with more than 100 rating counts: 1277

```

```

Number of products with more than 500 rating counts: 1129

```

```

Number of products with more than 1000 rating counts: 1047

```

```

Percentage of products with more than 100 rating counts 94.52%

```

Price range and discount insights

```

[20]: # Get min, max of the actual price column
min_actual_price = amazon['actual_price'].min()
max_actual_price = amazon['actual_price'].max()

print(f"Min actual price: {min_actual_price}")
print(f"Max actual price: {max_actual_price}")

```

```

Min actual price: 39.0

```

```

Max actual price: 139900.0

```

```
[21]: labels = ['Budget', 'Mid-Range', 'Premium', 'Luxury']

# Create the segment column
amazon['price_segment'] = pd.cut(amazon['actual_price'], bins = 4,
    ↪ labels=labels)

# See how many products fall into each category
print(amazon['price_segment'].value_counts())
```

```
price_segment
Budget      1426
Mid-Range    31
Premium       7
Luxury        1
Name: count, dtype: int64
```

```
[22]: # Get the average discount across categories
discount_by_category = amazon.groupby('category')['discount_percentage'].mean().
    ↪ sort_values(ascending=False).reset_index(name='Average Discount(%)')

discount_by_category.head(10)
```

```
[22]:
```

	category	Average Discount(%)
0	Computers&Accessories Accessories&Peripherals ...	90.0
1	Electronics Headphones,Earbuds&Accessories Ear...	90.0
2	Electronics Mobiles&Accessories MobileAccessor...	90.0
3	Electronics Headphones,Earbuds&Accessories Ada...	88.0
4	Computers&Accessories Accessories&Peripherals ...	87.5
5	Electronics Mobiles&Accessories MobileAccessor...	82.0
6	Computers&Accessories Components InternalHardD...	80.0
7	Computers&Accessories Accessories&Peripherals ...	78.5
8	Electronics Mobiles&Accessories MobileAccessor...	76.4
9	Electronics Headphones,Earbuds&Accessories Cases	76.0

```
[23]: # Pivot table showing average discount % for each price segment
discount_by_price_segment = amazon.pivot_table(index='price_segment',
    ↪ values='discount_percentage', aggfunc='mean', observed=False).
    ↪ sort_values(by='discount_percentage', ascending=False).reset_index()

print(discount_by_price_segment)
```

```
price_segment  discount_percentage
0      Budget      47.985273
1      Luxury      44.000000
2    Mid-Range      37.225806
3      Premium      34.714286
```

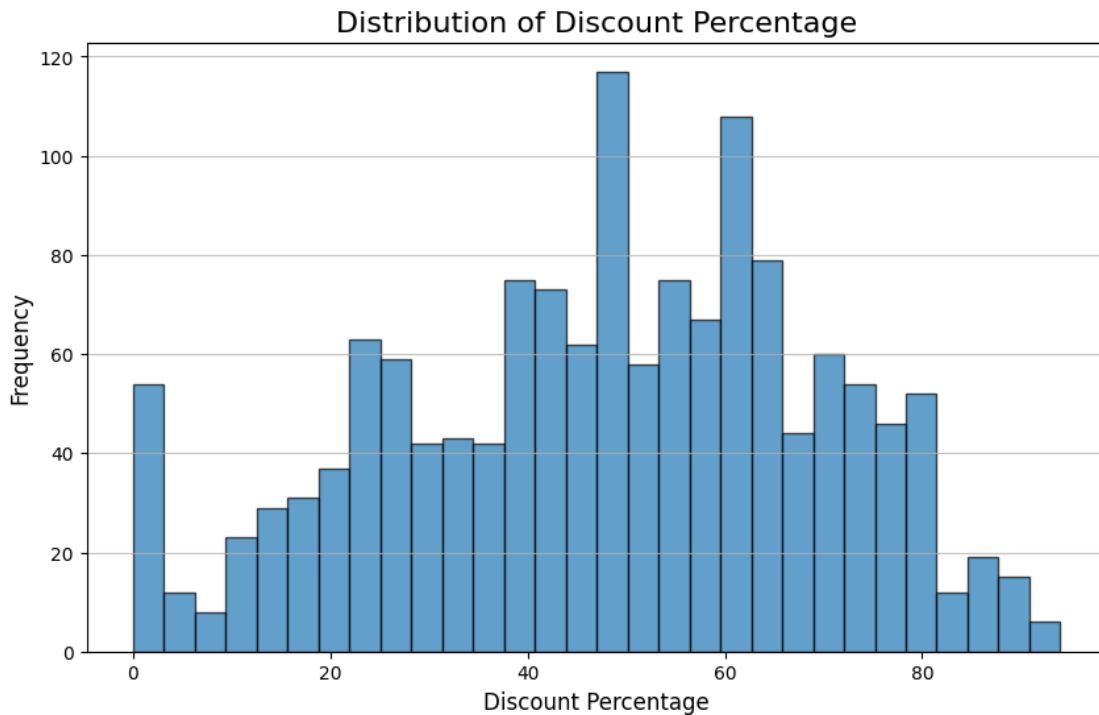
```
[24]: # Create a figure and an axes object
fig, ax = plt.subplots(figsize=(10, 6))

# Plot the histogram with more bins
ax.hist(amazon['discount_percentage'], bins=30, edgecolor='black', alpha=0.7);

# Add title and labels
ax.set_title('Distribution of Discount Percentage', fontsize=16);
ax.set_xlabel('Discount Percentage', fontsize=12);
ax.set_ylabel('Frequency', fontsize=12);

# Add a grid for better readability
ax.grid(axis='y', alpha=0.75);

# Display the plot
plt.show()
```



```
[25]: # Prepare the data for horizontal plot and multiline labels
# Make a copy to avoid modifying the original DataFrame if it's used elsewhere
plot_data = discount_by_category.head(10).copy()
plot_data['category'] = plot_data['category'].str.replace('|', '\n',
↪ regex=False)
```

```

# Set the figure size for better readability with horizontal bars
plt.figure(figsize=(12, 8))

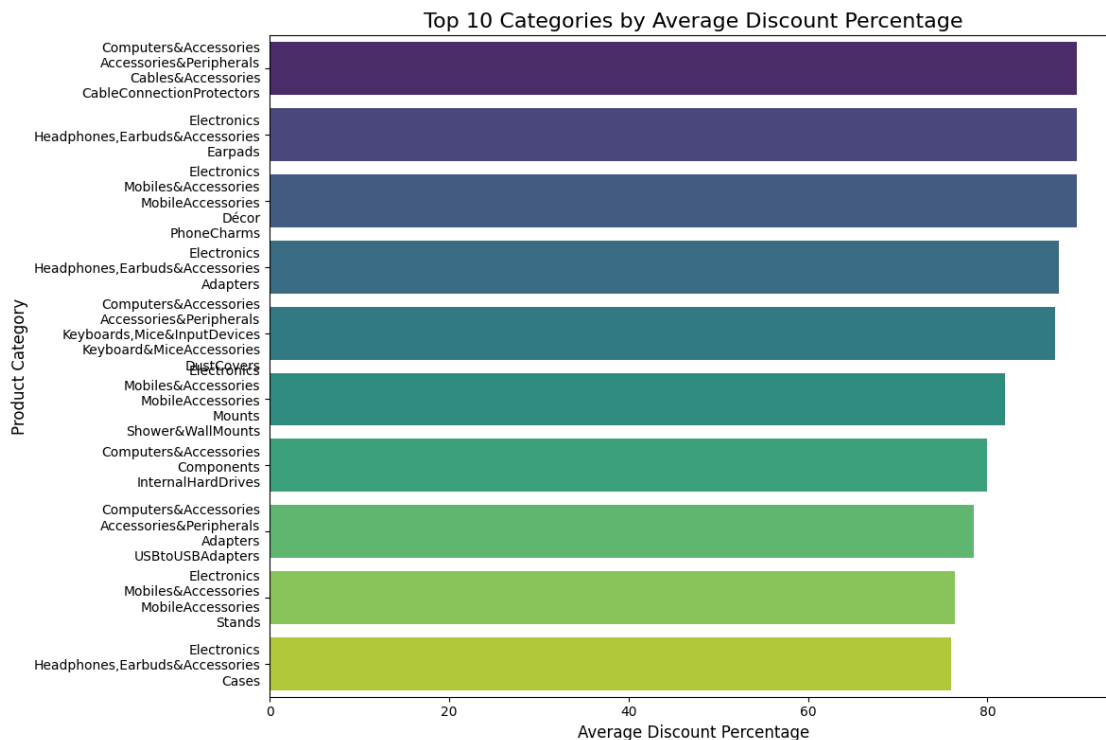
# Create a horizontal bar plot using seaborn
sns.barplot(x='Average Discount(%)', y='category', data=plot_data,
            hue='category', legend=False, palette='viridis')

# Add title and labels
plt.title('Top 10 Categories by Average Discount Percentage', fontsize=16)
plt.xlabel('Average Discount Percentage', fontsize=12)
plt.ylabel('Product Category', fontsize=12)

# Adjust layout to prevent labels from being cut off
plt.tight_layout()

# Display the plot
plt.show()

```



Parse categories into hierarchy levels

```

[26]: category_df = amazon['category'].str.split('|', expand=True)[[0,1]]

# Rename the columns dynamically based on the number of generated columns

```

```

category_df.columns = [f'category_level_{i+1}' for i in range(category_df.
↳shape[1])]

# Concatenate the new category columns with the original DataFrame
amazon = pd.concat([amazon, category_df], axis=1)

# Fill any NaN values in the new category level columns with 'Unknown'
for col in category_df.columns:
    amazon[col] = amazon[col].fillna('Unknown')

print("DataFrame after splitting 'category' column and filling NaNs:")
print("_" * 60)
display(amazon.head())

```

DataFrame after splitting 'category' column and filling NaNs:

```

-----
      product_id      product_name \
0  B07JW9H4J1  Wayona Nylon Braided USB to Lightning Fast Cha...
1  B098NS6PVG  Ambrane Unbreakable 60W / 3A Fast Charging 1.5...
2  B096MSW6CT  Sounce Fast Phone Charging Cable & Data Sync U...
3  B08HDJ86NZ  boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...
4  B08CF3B7N1  Portronics Konnect L 1.2M Fast Charging 3A 8 P...

      category  discounted_price \
0  Computers&Accessories|Accessories&Peripherals|...      399.0
1  Computers&Accessories|Accessories&Peripherals|...      199.0
2  Computers&Accessories|Accessories&Peripherals|...      199.0
3  Computers&Accessories|Accessories&Peripherals|...      329.0
4  Computers&Accessories|Accessories&Peripherals|...      154.0

      actual_price  discount_percentage  rating  rating_count \
0           1099.0             64.0      4.2         24269
1            349.0             43.0      4.0         43994
2           1899.0             90.0      3.9           7928
3            699.0             53.0      4.2         94363
4            399.0             61.0      4.2         16905

      about_product \
0  High Compatibility : Compatible With iPhone 12...
1  Compatible with all Type C enabled devices, be...
2  Fast Charger& Data Sync -With built-in safet...
3  The boAt Deuce USB 300 2 in 1 cable is compati...
4  [CHARGE & SYNC FUNCTION]- This cable comes wit...

      user_id \
0  AG3D604STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB...
1  AECPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX...

```


2 AGU3BBQ2V2DDAMOAKGFAWDDQ6QHA,AESFLDV2PT363T2AQ...
 3 AEWAZDZZJLQUYVOVGBEUKSLXHQ5A,AG5HTSFRRE6NL3M5S...
 4 AE3Q6KSUK5P75D5HFYHCRAOLODSA,AFUGIFH5ZAFXRDSZH...

user_name \

0 Manav,Adarsh gupta,Sundeep,S.Sayeed Ahmed,jasp...
 1 ArdKn,Nirbhay kumar,Sagar Viswanathan,Asp,Plac...
 2 Kunal,Himanshu,viswanath,sai niharka,saqib mal...
 3 Omkar dhale,JD,HEMALATHA,Ajwadh a.,amar singh ...
 4 rahuls6099,Swasat Borah,Ajay Wadke,Pranali,RVK...

review_id \

0 R3HXWTOLRPONMF,R2AJM3LFTLZHFO,R6AQJGUP6P86,R1K...
 1 RGIQEG07R9HS2,R1SMWZQ86XIN8U,R2J3Y1WL29GWDE,RY...
 2 R3J3EQQ9TZI5ZJ,R3E7WBGK7IDOKV,RWU79XKQ6I1QF,R2...
 3 R3EEUZKKK9J36I,R3HJVYCLYOY554,REDECAZ7AMPQC,R1...
 4 R1BP4L2HH9TFUP,R16PVJEXKV6QZS,R2UPDB81N66T4P,R...

review_title \

0 Satisfied,Charging is really fast,Value for mo...
 1 A Good Braided Cable for Your Type C Device,Go...
 2 Good speed for earlier versions,Good Product,W...
 3 Good product,Good one,Nice,Really nice product...
 4 As good as original,Decent,Good one for second...

review_content \

0 Looks durable Charging is fine tooNo complains...
 1 I ordered this cable to connect my phone to An...
 2 Not quite durable and sturdy,https://m.media-a...
 3 Good product,long wire,Charges good,Nice,I bou...
 4 Bought this instead of original apple, does th...

img_link \

0 https://m.media-amazon.com/images/W/WEBP_40237...
 1 https://m.media-amazon.com/images/W/WEBP_40237...
 2 https://m.media-amazon.com/images/W/WEBP_40237...
 3 https://m.media-amazon.com/images/I/41V5FtEWPk...
 4 https://m.media-amazon.com/images/W/WEBP_40237...

product_link price_segment \

0 https://www.amazon.in/Wayona-Braided-WN3LG1-Sy... Budget
 1 https://www.amazon.in/Ambrane-Unbreakable-Char... Budget
 2 https://www.amazon.in/Sounce-iPhone-Charging-C... Budget
 3 https://www.amazon.in/Deuce-300-Resistant-Tang... Budget
 4 https://www.amazon.in/Portronics-Konnnect-POR-1... Budget

category_level_1 category_level_2

0 Computers&Accessories Accessories&Peripherals

```

1 Computers&Accessories Accessories&Peripherals
2 Computers&Accessories Accessories&Peripherals
3 Computers&Accessories Accessories&Peripherals
4 Computers&Accessories Accessories&Peripherals

```

```

[27]: # Calculate z-score
amazon['rating_zscore'] = (amazon['rating'] - amazon['rating'].mean()) /
    ↪amazon['rating'].std()
outlier_count = amazon[amazon['rating_zscore'].abs() > 3].shape[0]

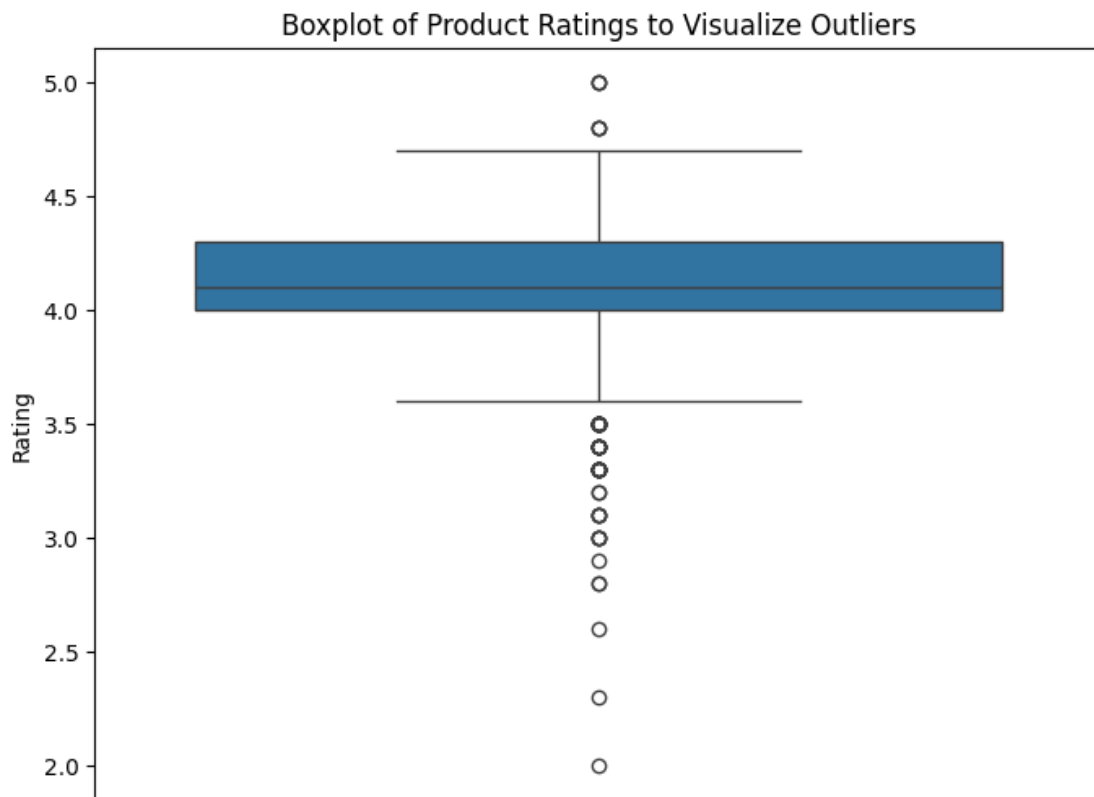
print(f"Number of outliers in the 'rating' column: {outlier_count}")

import matplotlib.pyplot as plt
import seaborn as sns

# Create a boxplot to visualize outliers
plt.figure(figsize=(8, 6))
sns.boxplot(y=amazon['rating'])
plt.title('Boxplot of Product Ratings to Visualize Outliers')
plt.ylabel('Rating')
plt.show()

```

Number of outliers in the 'rating' column: 19



0.2 Section B: Exploratory Data Analysis

```
[28]: # Get number of reviews per product
reviews_per_product = amazon.groupby('product_id')['review_id'].count().
    ↪sort_values(ascending=False).reset_index(name='Total Reviews')

# Merge it with the original dataset to get some relevant columns
merged_reviews_per_product = pd.merge(reviews_per_product,
    ↪amazon[['product_id', 'product_name', 'category_level_1',
    ↪'category_level_2', 'rating', 'discounted_price']], on='product_id',
    ↪how='left')

# Get top 10 most reviewed products
top_10_most_reviewed_products = merged_reviews_per_product.head(10)

top_10_most_reviewed_products
```

```
[28]:  product_id  Total Reviews  \
0  B09W5XR9RT              3
1  B09W5XR9RT              3
2  B09W5XR9RT              3
3  B08R69VDHT              3
4  B08R69VDHT              3
5  B08R69VDHT              3
6  B09NHVCHS9              3
7  B09NHVCHS9              3
8  B09NHVCHS9              3
9  B082T6V3DT              3

                                product_name  category_level_1  \
0  Duracell USB C To Lightning Apple Certified (M...  Computers&Accessories
1  Duracell USB C To Lightning Apple Certified (M...  Computers&Accessories
2  Duracell USB C To Lightning Apple Certified (M...  Computers&Accessories
3  Pinnacalz Original Combo of 2 Micro USB Fast Ch...  Computers&Accessories
4  Pinnacalz Original Combo of 2 Micro USB Fast Ch...  Computers&Accessories
5  Pinnacalz Original Combo of 2 Micro USB Fast Ch...  Computers&Accessories
6          Flix Micro Usb Cable For Smartphone (Black)  Computers&Accessories
7          Flix Micro Usb Cable For Smartphone (Black)  Computers&Accessories
8          Flix Micro Usb Cable For Smartphone (Black)  Computers&Accessories
9  AmazonBasics New Release Nylon USB-A to Lightn...  Computers&Accessories

    category_level_2  rating  discounted_price
0  Accessories&Peripherals    4.4           970.0
1  Accessories&Peripherals    4.4           970.0
2  Accessories&Peripherals    4.4           970.0
```

3	Accessories&Peripherals	4.0	115.0
4	Accessories&Peripherals	4.0	115.0
5	Accessories&Peripherals	4.0	115.0
6	Accessories&Peripherals	4.0	59.0
7	Accessories&Peripherals	4.0	59.0
8	Accessories&Peripherals	4.0	59.0
9	Accessories&Peripherals	4.3	799.0

Visualize top 10 categories by number of products.

```
[29]: # Get products per top level category
products_per_category = amazon.groupby('category_level_1')['product_id'].
    ↪unique().sort_values(ascending=False).reset_index(name="Total Products")
print("=" * 31)
print("Products per top level category")
print("=" * 31)
display(products_per_category.head(10))
print('\n')

# Get products per top level category
products_per_sub_category = amazon.groupby(['category_level_1',
    ↪'category_level_2'])['product_id'].unique().sort_values(ascending=False).
    ↪reset_index(name="Total Products")
print("=" * 48)
print("Products per combination of top & sub categories")
print("=" * 48)
products_per_sub_category.head(10)
```

```
=====
Products per top level category
=====
```

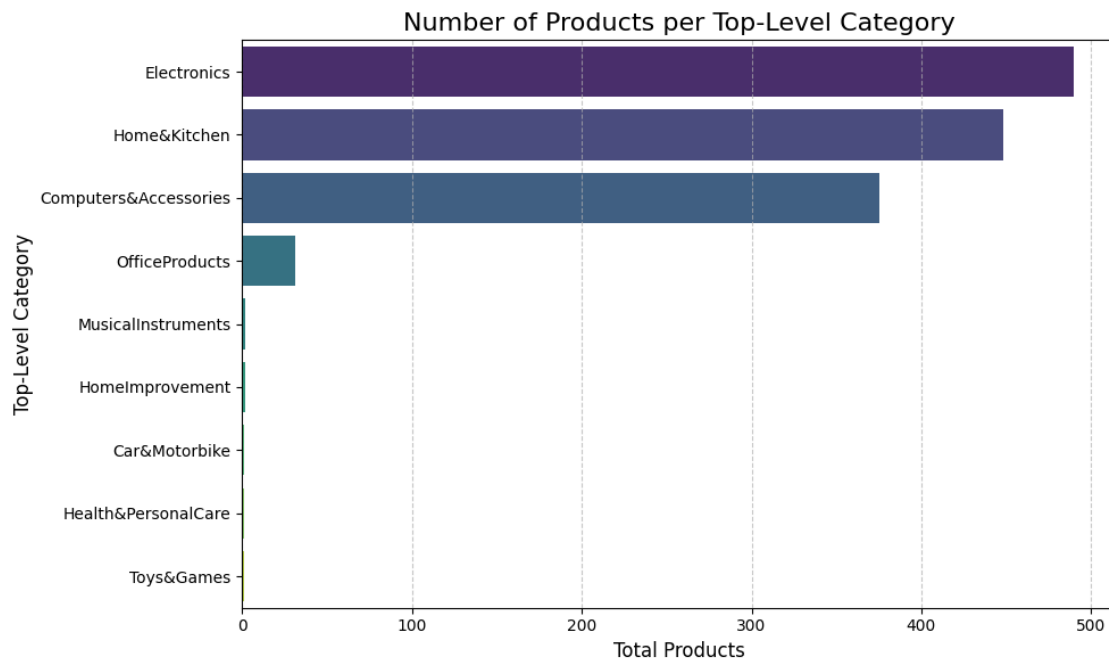
	category_level_1	Total Products
0	Electronics	490
1	Home&Kitchen	448
2	Computers&Accessories	375
3	OfficeProducts	31
4	MusicalInstruments	2
5	HomeImprovement	2
6	Car&Motorbike	1
7	Health&PersonalCare	1
8	Toys&Games	1

```
=====
Products per combination of top & sub categories
=====
```

```
[29]:
```

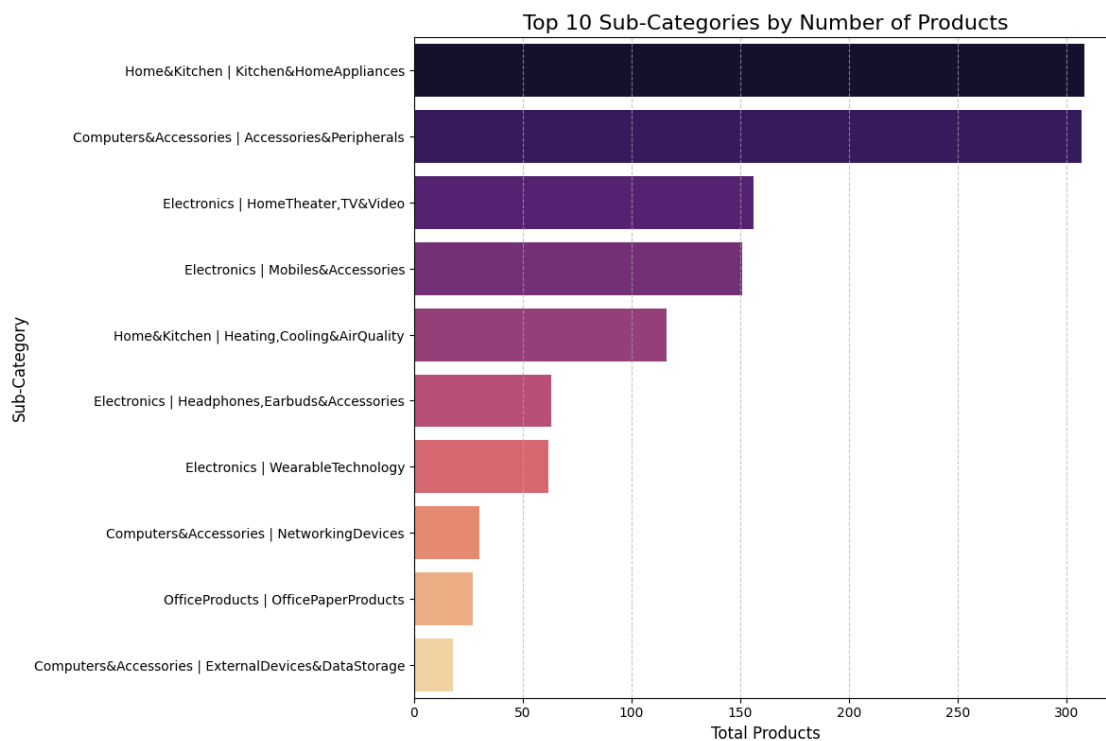
	category_level_1	category_level_2	Total Products
0	Home&Kitchen	Kitchen&HomeAppliances	308
1	Computers&Accessories	Accessories&Peripherals	307
2	Electronics	HomeTheater,TV&Video	156
3	Electronics	Mobiles&Accessories	151
4	Home&Kitchen	Heating,Cooling&AirQuality	116
5	Electronics	Headphones,Earbuds&Accessories	63
6	Electronics	WearableTechnology	62
7	Computers&Accessories	NetworkingDevices	30
8	OfficeProducts	OfficePaperProducts	27
9	Computers&Accessories	ExternalDevices&DataStorage	18

```
[30]: # Create a bar chart for products per top-level category
plt.figure(figsize=(10, 6))
sns.barplot(x='Total Products', y='category_level_1',
            data=products_per_category, palette='viridis', hue='category_level_1',
            legend=False)
plt.title('Number of Products per Top-Level Category', fontsize=16)
plt.xlabel('Total Products', fontsize=12)
plt.ylabel('Top-Level Category', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



```
[31]: # Combine category levels for a more descriptive label for the sub-category
      ↪ chart
products_per_sub_category['full_category'] =
    ↪ products_per_sub_category['category_level_1'] + ' | ' +
    ↪ products_per_sub_category['category_level_2']

# Create a bar chart for products per sub-category (top 10 for readability)
plt.figure(figsize=(12, 8))
sns.barplot(x='Total Products', y='full_category',
    ↪ data=products_per_sub_category.head(10), palette='magma',
    ↪ hue='full_category', legend=False)
plt.title('Top 10 Sub-Categories by Number of Products', fontsize=16)
plt.xlabel('Total Products', fontsize=12)
plt.ylabel('Sub-Category', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Average rating per category

```
[32]: # Get average rating per category
print("Average rating per category")
print("_" * 30)
```

```

avg_rating_per_category = amazon.groupby('category_level_1')['rating'].mean().
    ↪round(2).sort_values(ascending=False).reset_index(name='Average Rating')
display(avg_rating_per_category)
print('\n')

print("\nAverage rating per combination of category & sub category")
print("_" * 60)
avg_rating_per_sub_category = amazon.groupby(['category_level_1',
    ↪'category_level_2'])['rating'].mean().round(2).sort_values(ascending=False).
    ↪reset_index(name='Average Rating')

avg_rating_per_sub_category

```

Average rating per category

```

-----
      category_level_1  Average Rating
0      OfficeProducts      4.31
1      Toys&Games      4.30
2      HomeImprovement  4.25
3  Computers&Accessories  4.15
4      Electronics      4.08
5      Home&Kitchen      4.04
6  Health&PersonalCare  4.00
7  MusicalInstruments  3.90
8      Car&Motorbike      3.80

```

Average rating per combination of category & sub category

```

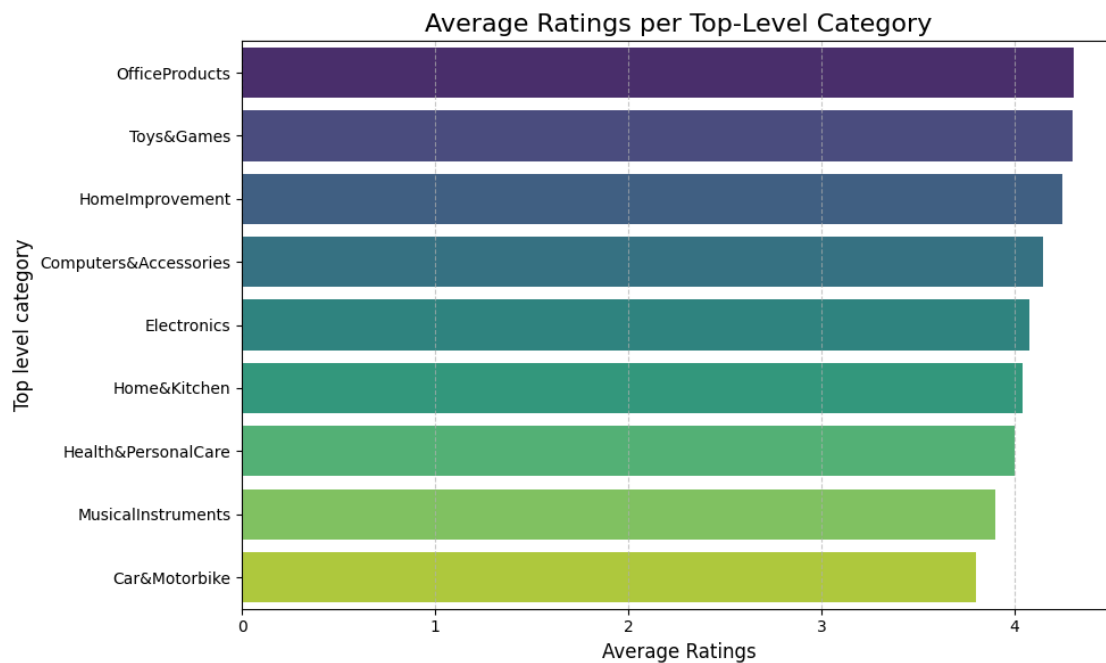
-----
[32]:      category_level_1      category_level_2 \
0  Computers&Accessories      Tablets
1      Electronics      PowerAccessories
2      OfficeProducts      OfficeElectronics
3  Computers&Accessories      Components
4      Electronics  GeneralPurposeBatteries&BatteryChargers
5      Electronics      Accessories
6      Home&Kitchen      CraftMaterials
7  Computers&Accessories  ExternalDevices&DataStorage
8      Toys&Games      Arts&Crafts
9      OfficeProducts      OfficePaperProducts
10  Computers&Accessories      Monitors
11      HomeImprovement      Electrical
12  Computers&Accessories      NetworkingDevices
13  Computers&Accessories  Accessories&Peripherals

```

14	Electronics	Mobiles&Accessories
15	Electronics	Cameras&Photography
16	Home&Kitchen	Kitchen&Dining
17	Electronics	HomeTheater,TV&Video
18	Electronics	HomeAudio
19	Home&Kitchen	Kitchen&HomeAppliances
20	Home&Kitchen	HomeStorage&Organization
21	Electronics	WearableTechnology
22	Health&PersonalCare	HomeMedicalSupplies&Equipment
23	Computers&Accessories	Laptops
24	Home&Kitchen	Heating,Cooling&AirQuality
25	Computers&Accessories	Printers,Inks&Accessories
26	Electronics	Headphones,Earbuds&Accessories
27	MusicalInstruments	Microphones
28	Car&Motorbike	CarAccessories

	Average Rating
0	4.60
1	4.50
2	4.45
3	4.38
4	4.35
5	4.34
6	4.34
7	4.32
8	4.30
9	4.29
10	4.25
11	4.25
12	4.16
13	4.15
14	4.13
15	4.12
16	4.10
17	4.08
18	4.07
19	4.05
20	4.03
21	4.02
22	4.00
23	4.00
24	3.99
25	3.96
26	3.93
27	3.90
28	3.80


```
[33]: # Create a bar chart for products per top-level category
plt.figure(figsize=(10, 6))
sns.barplot(x='Average Rating', y='category_level_1',
            data=avg_rating_per_category, palette='viridis', hue='category_level_1',
            legend=False)
plt.title('Average Ratings per Top-Level Category', fontsize=16)
plt.xlabel('Average Ratings', fontsize=12)
plt.ylabel('Top level category', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Discounts vs actual price correlation

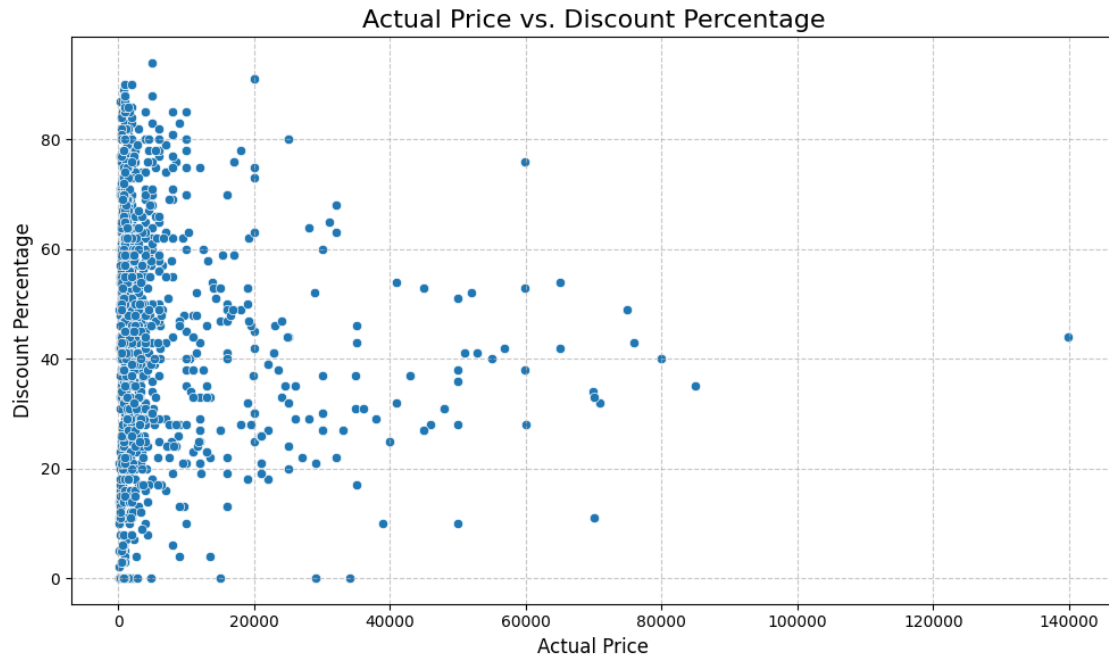
```
[34]: # Calculate the correlation between actual price and discount percentage
price_discount_corr = amazon['actual_price'].corr(amazon['discount_percentage'])

print(f"Correlation between actual price and discount percentage:
      {price_discount_corr:.2f}")
```

Correlation between actual price and discount percentage: -0.12

```
[35]: # Visualise the above correlation using a scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x='actual_price', y='discount_percentage', data=amazon)
```

```
plt.title('Actual Price vs. Discount Percentage', fontsize=16)
plt.xlabel('Actual Price', fontsize=12)
plt.ylabel('Discount Percentage', fontsize=12)
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.tight_layout()
```



User Engagement Insights

Which products have high ratings but low review counts?

```
[36]: # Filter products with high ratings
highly_rated_products = amazon[amazon['rating'] >= 4.5]

# For the highly rated products, count the reviews
highly_rated_products_total_reviews = highly_rated_products.
    ↪groupby('product_id')['review_id'].count().reset_index(name='Total Reviews')

# Merge with original dataset to get some useful columns from it
highly_rated_products_total_reviews_merged = pd.
    ↪merge(highly_rated_products_total_reviews, amazon[['product_id',
    ↪'product_name', 'category_level_1', 'category_level_2', 'rating']],
    ↪on='product_id', how='left')

# Finally, filter out only the highly rated but least reviewed products
```

```
highly_rated_products_least_reviewed =
↳highly_rated_products_total_reviews_merged[highly_rated_products_total_reviews_merged['Total
↳Reviews'] < 2]
```

```
highly_rated_products_least_reviewed
```

```
[36]:
```

	product_id	Total Reviews	\
0	B004I05BMQ	1	
1	B0083T2310	1	
2	B00K57MR22	1	
3	B00KIE28X0	1	
4	B00LHZWDOC	1	
..	...	...	
99	B0BP7XLX48	1	
100	B0BP89YBC1	1	
101	B0BQ3K23Y1	1	
102	B0BQRJ3C47	1	
103	B0BR4F878Q	1	

	product_name	category_level_1	\
0	Logitech M235 Wireless Mouse, 1000 DPI Optical...	Computers&Accessories	
1	Belkin Essential Series 4-Socket Surge Protect...	Electronics	
2	Sujata Dynamix DX Mixer Grinder, 900W, 3 Jars ...	Home&Kitchen	
3	Camel Artist Acrylic Color Box - 9ml Tubes, 12...	Home&Kitchen	
4	Luxor 5 Subject Single Ruled Notebook - A4, 70...	OfficeProducts	
..	...	...	
99	Syncwire LTG to USB Cable for Fast Charging Co...	Computers&Accessories	
100	Campfire Spring Chef Prolix Instant Portable W...	Home&Kitchen	
101	Oratech Coffee Frother electric, milk frother ...	Home&Kitchen	
102	REDTECH USB-C to Lightning Cable 3.3FT, [Apple...	Computers&Accessories	
103	Swiffer Instant Electric Water Heater Faucet T...	Home&Kitchen	

	category_level_2	rating
0	Accessories&Peripherals	4.5
1	PowerAccessories	4.5
2	Kitchen&HomeAppliances	4.6
3	CraftMaterials	4.5
4	OfficePaperProducts	4.5
..	...	...
99	Accessories&Peripherals	5.0
100	Heating,Cooling&AirQuality	4.7
101	Kitchen&HomeAppliances	4.8
102	Accessories&Peripherals	5.0
103	Heating,Cooling&AirQuality	4.8

```
[89 rows x 6 columns]
```

Are highly rated products also heavily reviewed?

```
[37]: highly_rated_products_total_reviews_merged[highly_rated_products_total_reviews_merged['Total_Reviews'] > 2]
```

```
[37]:
```

	product_id	Total Reviews	\
63	B09C6HXFC1	3	
64	B09C6HXFC1	3	
65	B09C6HXFC1	3	

	product_name	category_level_1	\
63	Duracell USB Lightning Apple Certified (Mfi) B...	Computers&Accessories	
64	Duracell USB Lightning Apple Certified (Mfi) B...	Computers&Accessories	
65	Duracell USB Lightning Apple Certified (Mfi) B...	Computers&Accessories	

	category_level_2	rating
63	Accessories&Peripherals	4.5
64	Accessories&Peripherals	4.5
65	Accessories&Peripherals	4.5

The highest rated products have only max 3 reviews.

Create 3 actionable insights for Amazon's product strategy based on EDA.

0.2.1 Detailed Actionable Insights for Amazon's Product Strategy

Based on the Exploratory Data Analysis, here are three high-impact strategies:

- 1. Monetize High-Satisfaction Niches (Tablets & Office Electronics):**
 - **Data Insight:** The 'Tablets' category holds the highest average rating (4.6), and 'Office Electronics' follows at 4.45.
 - **Action:** Amazon should prioritize inventory expansion and search visibility for these categories. High consumer satisfaction suggests these are low-risk areas for introducing Amazon-branded premium alternatives.
- 2. Optimize Discount Efficiency for High-Rated Electronics:**
 - **Data Insight:** There is a very weak correlation (-0.12) between actual price and discount percentage, yet many categories see 80-90% discounts.
 - **Action:** For products maintaining a rating above 4.2, Amazon can reduce extreme discounting (capping at 40-50%) without significantly impacting sales, as quality appears to be the primary driver of purchase satisfaction in these segments.
- 3. Bridge the 'Premium' Gap:**
 - **Data Insight:** 97% of the analyzed products are in the 'Budget' segment, while 'Mid-Range' and 'Premium' accounts for less than 3%.
 - **Action:** Launch a targeted campaign to convert 'Budget' customers in high-performing categories (like Computers & Accessories) to 'Mid-Range' options. The current lack of inventory in higher price tiers represents a significant missed opportunity for increasing Average Order Value (AOV).

0.3 Section C: Content-Based Filtering

A content-based recommendation system suggests items similar to those a user has liked in the past by analyzing item attributes (metadata like genre, director, description). It builds a user profile based on these features to predict preferences and recommend new, similar content. In our Amazon dataset, the best features to use are `product_name`, `category`, and `about_product`

Vectorize product text (`about_product` + `product_name`) using TF-IDF The standard way to perform content-based filtering is by converting that text into numbers using TF-IDF (Term Frequency-Inverse Document Frequency) and then measuring how “close” products are using Cosine Similarity.

First, we will combine the descriptive columns into a single “metadata” string that the computer can analyze.

```
[38]: # Remove duplicate product_id entries, keeping the first occurrence
amazon_unique_product_ids_df = amazon.drop_duplicates(subset='product_id',
↳keep='first').reset_index(drop=True)

# Combine relevant text features into one column
amazon_unique_product_ids_df['metadata'] =
↳amazon_unique_product_ids_df['product_name'] + ' ' +
↳amazon_unique_product_ids_df['category_level_1'] + ' ' +
↳amazon_unique_product_ids_df['category_level_2'] + ' ' +
↳amazon_unique_product_ids_df['about_product']

original_count = len(amazon['product_id'])
unique_count = len(amazon_unique_product_ids_df['product_id'])
print(f"Removed {original_count - unique_count} duplicate products.")
print("_" * 30)

amazon_unique_product_ids_df['metadata']
```

Removed 114 duplicate products.

```
[38]: 0      Wayona Nylon Braided USB to Lightning Fast Cha...
1      Ambrane Unbreakable 60W / 3A Fast Charging 1.5...
2      Sounce Fast Phone Charging Cable & Data Sync U...
3      boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...
4      Portronics Konnect L 1.2M Fast Charging 3A 8 P...

1346   Noir Aqua - 5pcs PP Spun Filter + 1 Spanner | ...
1347   Prestige Delight PRWO Electric Rice Cooker (1 ...
1348   Bajaj Majesty RX10 2000 Watts Heat Convector R...
1349   Havells Ventil Air DSP 230mm Exhaust Fan (Pist...
1350   Borosil Jumbo 1000-Watt Grill Sandwich Maker (...
Name: metadata, Length: 1351, dtype: object
```

Convert Text to Vectors (TF-IDF) Computers can't read, so we turn words into a mathematical matrix where unique/important words (like "Noise-Cancelling") get higher weights than common words (like "the").

```
[39]: from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize the vectorizer (removing common English "stop words")
tfidf = TfidfVectorizer(stop_words='english')

# Construct the TF-IDF matrix
tfidf_matrix = tfidf.fit_transform(amazon_unique_product_ids_df['metadata'])

print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
```

TF-IDF matrix shape: (1351, 10131)

Vectorize product text (about_product + product_name) using Embeddings Embeddings are the modern “gold standard” for recommendation systems, often outperforming TF-IDF because they understand semantics (meaning) rather than just matching keywords.

While TF-IDF treats words as isolated tokens, Embeddings represent them as dense vectors in a high-dimensional space.

Why Embeddings are “Smarter”

If one product description uses the word “Cinema” and another uses “Movie,” TF-IDF sees them as completely different. An Embedding model knows they are mathematically similar.

We will use a pre-trained model like Sentence-Transformers (built on BERT) to turn your Amazon product descriptions into 384 or 768-dimensional vectors.

```
[40]: from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import warnings
from transformers import logging

# Suppress the 'UNEXPECTED' / 'MISSING' library warnings to keep output clean
logging.set_verbosity_error()
warnings.filterwarnings('ignore', category=UserWarning)

# Load a pre-trained model (Semantic meaning of text)
model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

# Encode metadata column into vectors
# This creates a 'feature map' representing the semantic meaning of products
embeddings = model.encode(amazon_unique_product_ids_df['metadata'].tolist(),
    ↪ show_progress_bar=True)
```

modules.json: 0%| | 0.00/349 [00:00<?, ?B/s]

config_sentence_transformers.json: 0%| | 0.00/116 [00:00<?, ?B/s]

```

README.md: 0.00B [00:00, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/612 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/90.9M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/103 [00:00<?, ?it/s]
tokenizer_config.json: 0%|          | 0.00/350 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]
Batches: 0%|          | 0/43 [00:00<?, ?it/s]

```

Calculate Similarity Scores Now, we calculate the Cosine Similarity. This creates a “similarity map” where every product is compared to every other product. A score of 1.0 means they are identical; 0.0 means they share no keywords.

```

[41]: from sklearn.metrics.pairwise import cosine_similarity

# Compute the similarity scores for TF-IDF
tf_idf_cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

tf_idf_cosine_sim

[41]: array([[1.          , 0.14083543, 0.3124759 , ..., 0.02867665, 0.03686226,
        0.00205068],
        [0.14083543, 1.          , 0.15479607, ..., 0.          , 0.01218295,
        0.02306526],
        [0.3124759 , 0.15479607, 1.          , ..., 0.01962469, 0.03379857,
        0.          ],
        ...,
        [0.02867665, 0.          , 0.01962469, ..., 1.          , 0.15128859,
        0.02111193],
        [0.03686226, 0.01218295, 0.03379857, ..., 0.15128859, 1.          ,
        0.03016754],
        [0.00205068, 0.02306526, 0.          , ..., 0.02111193, 0.03016754,
        1.          ]])

```

```

[42]: from sklearn.metrics.pairwise import cosine_similarity

# Compute the similarity scores for Embeddings
embeddings_cosine_sim = cosine_similarity(embeddings, embeddings)

```

```
embeddings_cosine_sim
```

```
[42]: array([[0.9999998 , 0.5928134 , 0.83577853, ..., 0.03337806, 0.09149186,
          0.13132082],
          [0.5928134 , 1.0000002 , 0.62836325, ..., 0.16496398, 0.09005775,
          0.19492638],
          [0.83577853, 0.62836325, 1.0000001 , ..., 0.0303296 , 0.08821952,
          0.18403828],
          ...,
          [0.03337806, 0.16496398, 0.0303296 , ..., 1.          , 0.38171086,
          0.3244709 ],
          [0.09149186, 0.09005775, 0.08821952, ..., 0.38171086, 0.9999999 ,
          0.25040865],
          [0.13132082, 0.19492638, 0.18403828, ..., 0.3244709 , 0.25040865,
          0.99999964]], dtype=float32)
```

Recommendation Function This function takes a `product_id`, finds its index, and pulls the top 5 most similar items.

```
[43]: def get_recommendations(product_id, cosine_sim, get_hybrid_score=False):
      # Get the index and metadata for the base product
      base_product =
      ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'] ==
      ↪product_id].iloc[0]
      idx = base_product.name
      base_subcat = base_product['category_level_2']

      # Get pairwise similarity scores
      sim_scores = list(enumerate(cosine_sim[idx]))

      # Apply Hybrid Scoring if requested
      if get_hybrid_score:
          weighted_sim_scores = [(x, y * 0.7) for x, y in sim_scores]
          amazon['norm_rating'] = (amazon['rating'] - min_rating) / (max_rating -
          ↪min_rating)
          weighted_norm_ratings = [(x, amazon.loc[x, 'norm_rating'] * 0.3) for x, y
          ↪in sim_scores]
          sim_scores = [(s[0], s[1] + r[1]) for s, r in zip(weighted_sim_scores,
          ↪weighted_norm_ratings)]

      # Sort and get top 5 (excluding itself)
      sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

      # Get the scores of the 5 most similar products (excluding itself)
      sim_scores = sim_scores[1:6]

      # Get the product indices
```



```

final_indices = [i[0] for i in sim_scores]

return amazon_unique_product_ids_df[['product_name', 'category_level_1',
↪ 'category_level_2', 'actual_price', 'rating']].loc[final_indices]

```

Testing the recommendation function for TF-IDF

```

[44]: # Get product description for product id 'B09C6HXFC1'

# Print product name
base_product = amazon.loc[amazon['product_id'] == 'B09C6HXFC1',
↪ ['product_name']].drop_duplicates()
base_product.columns = ['']
print(f"Product Name => {base_product.values[0][0]}", '\n')

print("_" * 50)
print("\nTop 5 most similar products to the above product")
print("_" * 50, '\n')

# Get product recs for the base product
product_recs = get_recommendations('B09C6HXFC1', tf_idf_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual
↪ Price', 'Rating']
display(product_recs)

```

Product Name => Duracell USB Lightning Apple Certified (Mfi) Braided Sync & Charge Cable For Iphone, Ipad And Ipod. Fast Charging Lightning Cable, 3.9 Feet (1.2M) - Black

Top 5 most similar products to the above product

	Product Name	Category \
28	Duracell USB C To Lightning Apple Certified (M...	Computers&Accessories
194	Duracell Micro USB 3A Braided Sync & Fast Char...	Computers&Accessories
39	Duracell Type C To Type C 5A (100W) Braided Sy...	Computers&Accessories
149	Duracell Type-C To Micro 1.2M braided Sync & C...	Computers&Accessories
156	Belkin Apple Certified Lightning To Type C Cab...	Computers&Accessories

	Sub Category	Actual Price	Rating
28	Accessories&Peripherals	1999.0	4.4
194	Accessories&Peripherals	599.0	4.1
39	Accessories&Peripherals	1999.0	4.2
149	Accessories&Peripherals	699.0	4.2
156	Accessories&Peripherals	1999.0	4.4

Testing the recommendation function for Embeddings

```
[45]: # Get product description for product id 'B09C6HXFC1'

# Print product name
base_product = amazon.loc[amazon['product_id'] == 'B09C6HXFC1',
↪['product_name']].drop_duplicates()
base_product.columns = ['']
print(f"Product Name => {base_product.values[0][0]}", '\n')

print("_" * 50)
print("\nTop 5 most similar products to the above product")
print("_" * 50, '\n')

# Get product recs for the base product
product_recs = get_recommendations('B09C6HXFC1', embeddings_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
↪Price', 'Rating']
display(product_recs)
```

Product Name => Duracell USB Lightning Apple Certified (Mfi) Braided Sync & Charge Cable For Iphone, Ipad And Ipod. Fast Charging Lightning Cable, 3.9 Feet (1.2M) - Black

Top 5 most similar products to the above product

	Product Name	Category \
28	Duracell USB C To Lightning Apple Certified (M...	Computers&Accessories
194	Duracell Micro USB 3A Braided Sync & Fast Char...	Computers&Accessories
191	Amazon Basics USB A to Lightning MFi Certified...	Computers&Accessories
259	Amazon Basics USB A to Lightning PVC Molded Ny...	Computers&Accessories
238	Hi-Mobiler iPhone Charger Lightning Cable,2 Pa...	Computers&Accessories

	Sub Category	Actual Price	Rating
28	Accessories&Peripherals	1999.0	4.4
194	Accessories&Peripherals	599.0	4.1
191	Accessories&Peripherals	1999.0	4.1
259	Accessories&Peripherals	1999.0	4.2
238	Accessories&Peripherals	799.0	4.0

Recommend top 5 similar products to: A new product with no reviews

[46]:

```

# Get products with only 1 review as there are no products in this dataset with
↳no reviews
low_reviews_products = reviews_per_product[reviews_per_product['Total Reviews']
↳== 1]

low_rating_count_products = product_rating_count[product_rating_count['Total
↳Ratings'] < 10]

low_rating_count_low_review_products = pd.merge(low_reviews_products,
↳low_rating_count_products, on='product_id', how='inner').
↳sort_values(by='Total Ratings')

low_rating_count_low_review_products

```

```

[46]:

```

	product_id	Total Reviews	Total Ratings
0	BOB94JPY2N	1	0
8	BOBQRJ3C47	1	0
9	BOBPJBTB3F	1	2
5	BOB3JSWG81	1	2
3	BOBGPN4GGH	1	4
12	BOBNLFQDG2	1	4
2	BOBL3R4RGS	1	4
11	BOBP7XLX48	1	5
7	BOBN2576GQ	1	6
10	BOBNDD9TN6	1	7
1	BOB4SJKRDF	1	7
6	B09ZVJXN5L	1	8
4	B09P1MFKG1	1	9

```

[47]: # Get product recommendations for a new product with low reviews and low rating
↳counts, using TF-IDF

# Get product id
low_review_low_rating_count_product_id = low_rating_count_low_review_products.
↳iloc[0]['product_id']

# Print product name
base_product = amazon.loc[amazon['product_id'] ==
↳low_review_low_rating_count_product_id, ['product_name']].drop_duplicates()
base_product.columns = ['']
print(f"Product Name => {base_product.values[0][0]}", '\n')

print("_" * 60)
print("\nTop 5 most similar products to the above product using TF-IDF")
print("_" * 60, '\n')

# Get product recs for the base product

```

```
product_recs = get_recommendations(low_review_low_rating_count_product_id,
    ↪tf_idf_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
    ↪Price', 'Rating']
display(product_recs)
```

Product Name => Amazon Brand - Solimo 65W Fast Charging Braided Type C to C Data Cable | Suitable For All Supported Mobile Phones (1 Meter, Black)

Top 5 most similar products to the above product using TF-IDF

	Product Name	Category \
313	Amazon Brand - Solimo Fast Charging Braided Ty...	Computers&Accessories
177	Zoul USB C to USB C Fast Charging Cable 65W Ty...	Computers&Accessories
277	Amazon Brand - Solimo 3A Fast Charging Tough T...	Computers&Accessories
140	Zoul Type C to Type C Fast Charging Cable 65W ...	Computers&Accessories
71	CEDO 65W OnePlus Dash Warp Charge Cable, USB A...	Computers&Accessories

	Sub Category	Actual Price	Rating
313	Accessories&Peripherals	599.0	4.1
177	Accessories&Peripherals	1099.0	4.1
277	Accessories&Peripherals	299.0	3.8
140	Accessories&Peripherals	1099.0	4.1
71	Accessories&Peripherals	599.0	4.1

```
[48]: # Get product recommendations for a new product with low reviews and low rating_
    ↪counts, using Embeddings

print(f"Product Name => {base_product.values[0][0]}", '\n')

print("_" * 65)
print("\nTop 5 most similar products to the above product using Embeddings")
print("_" * 65, '\n')

# Get product recs for the base product
product_recs = get_recommendations(low_review_low_rating_count_product_id,
    ↪embeddings_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
    ↪Price', 'Rating']
display(product_recs)
```

Product Name => Amazon Brand - Solimo 65W Fast Charging Braided Type C to C Data Cable | Suitable For All Supported Mobile Phones (1 Meter, Black)

Top 5 most similar products to the above product using Embeddings

	Product Name	Category \
313	Amazon Brand - Solimo Fast Charging Braided Ty...	Computers&Accessories
241	Wayona Usb Type C To Usb Nylon Braided Quick C...	Computers&Accessories
161	Wayona Type C Cable Nylon Braided USB C QC 3.0...	Computers&Accessories
277	Amazon Brand - Solimo 3A Fast Charging Tough T...	Computers&Accessories
71	CEDO 65W OnePlus Dash Warp Charge Cable, USB A...	Computers&Accessories

	Sub Category	Actual Price	Rating
313	Accessories&Peripherals	599.0	4.1
241	Accessories&Peripherals	999.0	4.3
161	Accessories&Peripherals	999.0	4.3
277	Accessories&Peripherals	299.0	3.8
71	Accessories&Peripherals	599.0	4.1

Recommend top 5 similar products to: A product with high user dropout (bad ratings)

```
[49]: # Get product recommendations for the lowest rated product, using TF-IDF

# Filter products with low/bad ratings and get the lowest rated product from it
lowest_rated_product = amazon[amazon['rating'] < 3].sort_values(by='rating').
    ↪iloc[0]

# Get product id
lowest_rated_product_id = lowest_rated_product['product_id']

# Print product name
lowest_rated_product_name = lowest_rated_product['product_name']
print(f"Product Name => {lowest_rated_product_name}", '\n')

print("_" * 60)
print("\nTop 5 most similar products to the above product using TF-IDF")
print("_" * 60, '\n')

# Get product recs for the base product
product_recs = get_recommendations(lowest_rated_product_id, tf_idf_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
    ↪Price', 'Rating']
display(product_recs)
```

Product Name => Khaitan ORFin Fan heater for Home and kitchen-K0 2215

Top 5 most similar products to the above product using TF-IDF

	Product Name	Category	\	
921	Orpat OEH-1260 2000-Watt Fan Heater (Grey)	Home&Kitchen		
975	Amazon Basics 2000/1000 Watt Room Heater with ...	Home&Kitchen		
1178	!!HANEUL!!1000 Watt/2000-Watt Room Heater!! Fa...	Home&Kitchen		
1044	!!1000 Watt/2000-Watt Room Heater!! Fan Heater...	Home&Kitchen		
1246	Kenstar 2400 Watts 9 Fins Oil Filled Radiator ...	Home&Kitchen		
	Sub Category	Actual Price	Rating	
921	Heating,Cooling&AirQuality	1650.0	4.1	
975	Heating,Cooling&AirQuality	1699.0	3.1	
1178	Heating,Cooling&AirQuality	1599.0	3.4	
1044	Heating,Cooling&AirQuality	1599.0	4.5	
1246	Heating,Cooling&AirQuality	11990.0	3.9	

As seen above, the problem with a pure content-based search is that it might find a product that is “technically” a 99% match but has a 1-star rating (or some other poor rating). By incorporating ratings, you ensure that you are recommending products that are not only relevant but also high-quality. This is known as a Hybrid Scoring or Rank Fusion approach.

We can create a new “Hybrid Score” that is a weighted average of the Similarity Score and the Normalized Rating. The `get_recommendations` function accepts a third argument as a boolean, which when set to `True` (`False` by default) will generate a hybrid score.

```
[50]: # Get product recommendations for the lowest rated product, using TF-IDF with
      ↪hybrid scores

print(f"Product Name => {lowest_rated_product_name}", '\n')

print("_" * 75)
print("\nTop 5 most similar products to the above product (TF-IDF with Hybrid
      ↪Score)")
print("_" * 75, '\n')

# Get product recs for the base product
product_recs = get_recommendations(lowest_rated_product_id, tf_idf_cosine_sim,
      ↪True)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual
      ↪Price', 'Rating']
display(product_recs)
```

Product Name => Khaitan ORFin Fan heater for Home and kitchen-K0 2215

Top 5 most similar products to the above product (TF-IDF with Hybrid Score)

	Product Name	Category \
1226	SaiEllin Room Heater For Home 2000 Watts Room ...	Home&Kitchen
1044	!!1000 Watt/2000-Watt Room Heater!! Fan Heater...	Home&Kitchen
1094	Havells OFR 13 Wave Fin with PTC Fan Heater 29...	Home&Kitchen
1245	KHAITAN AVAANTE KA-2013 1200 Watt 3-Rod Haloge...	Home&Kitchen
970	Crompton Arno Neo 15-L 5 Star Rated Storage Wa...	Home&Kitchen

	Sub Category	Actual Price	Rating
1226	Heating,Cooling&AirQuality	1999.0	3.9
1044	Heating,Cooling&AirQuality	1599.0	4.5
1094	Heating,Cooling&AirQuality	19825.0	4.1
1245	Heating,Cooling&AirQuality	3999.0	3.5
970	Heating,Cooling&AirQuality	10400.0	4.1

[51]: *# Get product recommendations for the lowest rated product, using Embeddings*

```
print(f"Product Name => {lowest_rated_product_name}", '\n')

print("_" * 60)
print("\nTop 5 most similar products to the above product (Embeddings)")
print("_" * 60, '\n')

# Get product recs for the base product
product_recs = get_recommendations(lowest_rated_product_id,
    ↪embeddings_cosine_sim)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
    ↪Price', 'Rating']
display(product_recs)
```

Product Name => Khaitan ORFin Fan heater for Home and kitchen-K0 2215

Top 5 most similar products to the above product (Embeddings)

	Product Name	Category \
921	Orpat OEH-1260 2000-Watt Fan Heater (Grey)	Home&Kitchen
1245	KHAITAN AVAANTE KA-2013 1200 Watt 3-Rod Haloge...	Home&Kitchen
1179	Melbon VM-905 2000-Watt Room Heater (ISI Certi...	Home&Kitchen
975	Amazon Basics 2000/1000 Watt Room Heater with ...	Home&Kitchen
1178	!!HANEUL!!1000 Watt/2000-Watt Room Heater!! Fa...	Home&Kitchen

	Sub Category	Actual Price	Rating
921	Heating,Cooling&AirQuality	1650.0	4.1
1245	Heating,Cooling&AirQuality	3999.0	3.5
1179	Heating,Cooling&AirQuality	2999.0	4.6
975	Heating,Cooling&AirQuality	1699.0	3.1
1178	Heating,Cooling&AirQuality	1599.0	3.4

```
[52]: # Get product recommendations for the lowest rated product, using Embeddings
      ↪with hybrid scores
```

```
print(f"Product Name => {lowest_rated_product_name}", '\n')

print("_" * 80)
print("\nTop 5 most similar products to the above product (Embeddings with
      ↪Hybrid Score)")
print("_" * 80, '\n')

# Get product recs for the base product
product_recs = get_recommendations(lowest_rated_product_id,
      ↪embeddings_cosine_sim, True)
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual
      ↪Price', 'Rating']
display(product_recs)
```

Product Name => Khaitan ORFin Fan heater for Home and kitchen-K0 2215

Top 5 most similar products to the above product (Embeddings with Hybrid Score)

	Product Name	Category \
1179	Melbon VM-905 2000-Watt Room Heater (ISI Certi...	Home&Kitchen
1245	KHAITAN AVAANTE KA-2013 1200 Watt 3-Rod Haloge...	Home&Kitchen
1044	!!1000 Watt/2000-Watt Room Heater!! Fan Heater...	Home&Kitchen
921	Orpat OEH-1260 2000-Watt Fan Heater (Grey)	Home&Kitchen
999	USHA Heat Convector 812 T 2000-Watt with Insta...	Home&Kitchen

	Sub Category	Actual Price	Rating
1179	Heating,Cooling&AirQuality	2999.0	4.6
1245	Heating,Cooling&AirQuality	3999.0	3.5
1044	Heating,Cooling&AirQuality	1599.0	4.5
921	Heating,Cooling&AirQuality	1650.0	4.1
999	Heating,Cooling&AirQuality	2990.0	3.8

Add category, price, and discount to enhance content vectors To create a truly powerful “Hybrid” recommendation system, we will combine our high-dimensional content vectors (text) with our metadata features (category, price, and discount).

This prevents the system from recommending a \$1,000 product when the user is looking at a \$20 “budget” item, even if the descriptions are similar.

1. Pre-processing Metadata Numerical data must be scaled so that large numbers (like a \$5,000 price) don’t overpower smaller numbers (like a 4.5 rating).

- Category: Convert to “One-Hot Encodings” (0s and 1s).
- Price & Discount: Use Min-Max Scaling to bring them into a 0–1 range.

```
[53]: from sklearn.preprocessing import MinMaxScaler, OneHotEncoder

# Convert category columns to one-hot encoding
encoder = OneHotEncoder(sparse_output=False)
encoded_categories = encoder.
    ↪fit_transform(amazon_unique_product_ids_df[['category_level_1',
    ↪'category_level_2']])

# Scale numerical features
scaler = MinMaxScaler()
scaled_features = scaler.
    ↪fit_transform(amazon_unique_product_ids_df[['actual_price',
    ↪'discount_percentage', 'rating']])

encoded_categories.shape, scaled_features.shape
```

```
[53]: ((1351, 38), (1351, 3))
```

2. Combining Vectors (Feature Fusion) We can connect our text embeddings and numerical features together using `np.hstack`. We can multiply specific features by a weight if we want them to be more influential.

```
[54]: # Increase the weight of Category and Price to ensure relevance
# We multiply categorical features by 5 to force recommendations within the
    ↪same category
weighted_categories = encoded_categories * 5.0

# We give Price a weight of 2.0 to keep it in a similar bracket
scaled_features[:, 0] *= 2.0

# Combine everything into one "Master Vector"
# Text embeddings (384 dims) + Weighted Categories + Scaled Numerical Features
combined_vectors = np.hstack((embeddings, weighted_categories, scaled_features))

print(f"New combined vector shape: {combined_vectors.shape}")
```

New combined vector shape: (1351, 425)

4. Refining the Search Now that the vectors are combined, we calculate Cosine Similarity just like before. The difference is that now the “distance” between products accounts for their price bracket and category.

```
[55]: # Re-calculate similarity with the new weighted vectors

# Check for NaNs and fill them with 0 to prevent the ValueError
if np.isnan(combined_vectors).any():
    combined_vectors = np.nan_to_num(combined_vectors)

# Calculate cosine similarity for combined vectors
combined_vectors_cosine_sim = cosine_similarity(combined_vectors,
↪combined_vectors)

print("Similarity matrix updated with categorical weighting.\n")
combined_vectors_cosine_sim
```

Similarity matrix updated with categorical weighting.

```
[55]: array([[1.          , 0.99162355, 0.9960257 , ..., 0.0121491 , 0.01485871,
        0.01647301],
       [0.99162355, 1.          , 0.99045158, ..., 0.01275324, 0.012816  ,
        0.01574712],
       [0.9960257 , 0.99045158, 1.          , ..., 0.0126235 , 0.01494594,
        0.01721916],
       ...,
       [0.0121491 , 0.01275324, 0.0126235 , ..., 1.          , 0.98780335,
        0.50097743],
       [0.01485871, 0.012816  , 0.01494594, ..., 0.98780335, 1.          ,
        0.50069641],
       [0.01647301, 0.01574712, 0.01721916, ..., 0.50097743, 0.50069641,
        1.          ]])
```

```
[56]: # Get product recommendations for the lowest rated product, using Combined
↪Vectors

print(f"Product Name => {lowest_rated_product_name}", '\n')

print("_" * 40)
print("\nTop 5 Similar Products (Combined Vectors)")
print("_" * 40, '\n')

# Calling with strict_category=True to ensure we only get heaters/fans
product_recs = get_recommendations(lowest_rated_product_id,
↪combined_vectors_cosine_sim)
```

```
product_recs.columns = ['Product Name', 'Category', 'Sub Category', 'Actual_
↪Price', 'Rating']
display(product_recs)
```

Product Name => Khaitan ORFin Fan heater for Home and kitchen-K0 2215

Top 5 Similar Products (Combined Vectors)

	Product Name	Category \
975	Amazon Basics 2000/1000 Watt Room Heater with ...	Home&Kitchen
1245	KHAITAN AVAANTE KA-2013 1200 Watt 3-Rod Haloge...	Home&Kitchen
1178	!!HANEUL!!1000 Watt/2000-Watt Room Heater!! Fa...	Home&Kitchen
1348	Bajaj Majesty RX10 2000 Watts Heat Convector R...	Home&Kitchen
1104	Eopora PTC Ceramic Fast Heating Room Heater fo...	Home&Kitchen

	Sub Category	Actual Price	Rating
975	Heating,Cooling&AirQuality	1699.0	3.1
1245	Heating,Cooling&AirQuality	3999.0	3.5
1178	Heating,Cooling&AirQuality	1599.0	3.4
1348	Heating,Cooling&AirQuality	3080.0	3.6
1104	Heating,Cooling&AirQuality	2999.0	3.3

Evaluate recommendations: How diverse and relevant are the content-based results?

1. TF-IDF (Term Frequency-Inverse Document Frequency)

- **Relevance:** High keyword matching. It is excellent at finding products with identical technical specifications (e.g., matching ‘65W’, ‘Braided’, ‘Type C’).
- **Diversity:** Low. It tends to stay very close to the specific terminology used in the base product, often missing similar items that use different synonyms.

2. Semantic Embeddings (all-MiniLM-L6-v2)

- **Relevance:** Good. It understands context (e.g., recognizing that ‘Cinema’ and ‘Movie’ are related). It is less prone to failing if a keyword is missing but the intent is the same.
- **Diversity:** Higher. It pulls in a broader range of products that are conceptually similar, though sometimes it can be too broad, occasionally pulling in items from different price brackets or sub-categories if the descriptions are stylistically similar.

3. Combined Vectors (Feature Fusion)

- **Relevance:** Excellent. By weighting categories and scaling prices, this method ensures that a ‘Budget’ heater doesn’t get a ‘Premium’ AC unit as a recommendation just because both mention ‘heating/cooling’.

- **Diversity:** Balanced. It provides the most “logical” recommendations for a real-world store, maintaining semantic similarity while respecting product hierarchy and user expectations regarding price.

Summary Evaluation For this Amazon dataset, **Combined Vectors** provide the most robust recommendations. While **Embeddings** are superior for capturing human-like meaning, the addition of hard metadata (Price & Category) in the Combined approach is essential to prevent ‘logical’ errors (like price-mismatches). **TF-IDF** remains a useful baseline for strict technical attribute matching but lacks the ‘intelligence’ needed for modern discovery.

0.4 Section D: Collaborative Filtering (User–Item) :

Create a user-item matrix using `user_id`, `product_id`, and `rating`.

```
[57]: # Create a user-item matrix
# We use pivot_table to handle cases where there might be duplicate entries
# (though we've cleaned many)
user_item_matrix = amazon.pivot_table(index='user_id', columns='product_id',
    values='rating')

print(f"User-Item Matrix Shape: {user_item_matrix.shape}")
print(f"Number of Users: {user_item_matrix.shape[0]}")
print(f"Number of Products: {user_item_matrix.shape[1]}")

# Display a snippet of the matrix
display(user_item_matrix.head())

# Calculate Sparsity
total_elements = user_item_matrix.size
non_zero_elements = user_item_matrix.notna().sum().sum()
sparsity = (1 - (non_zero_elements / total_elements)) * 100
print(f"Matrix Sparsity: {round(sparsity, 2)}%")
```

User-Item Matrix Shape: (1193, 1350)

Number of Users: 1193

Number of Products: 1350

product_id	B002PD61Y4	B002SZEOLG	\
user_id			
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEYO2IJ5I5GDWZAH...	NaN	NaN	
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQOTEGUMS6...	NaN	NaN	
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN	
AE27U0ZENYSWCQVQRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN	
AE2JTMRKU0IIVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN	

product_id	B003B00484	B003L62T7W	\
user_id			
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEYO2IJ5I5GDWZAH...	NaN	NaN	
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQOTEGUMS6...	NaN	NaN	
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN	

AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	B004I05BMQ	B005FYNT3G \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	B005LJQMCK	B005LJQMZC \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	B006LW0WDQ	B0073QGKAS \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	... B0BP18W8TM \	
user_id	...	
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	...	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	...	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	...	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	...	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	...	NaN
product_id	B0BP7XLX48	B0BP89YBC1 \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKU0IVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	B0BPBG712X	B0BPBXNQQT \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN

AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKITUOIVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	BOBPCJM7TB	BOBPJBTB3F \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEYO2IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKITUOIVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	BOBQ3K23Y1	BOBQRJ3C47 \
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEYO2IJ5I5GDWZAH...	NaN	NaN
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	NaN
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	NaN
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	NaN
AE2JTMRKITUOIVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	NaN
product_id	BOBR4F878Q	
user_id		
AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEYO2IJ5I5GDWZAH...	NaN	
AE23RS3W7GZO7LHYKJU6KSKVM4MQ,AEQUNEY6GQ0TEGUMS6...	NaN	
AE242TR3GQ6TYC6W4SJ5UYYKBTYQ	NaN	
AE27UOZENYSWCQVQRRUQIV2ZM7VA,AGMYSLV6NNOAYES25J...	NaN	
AE2JTMRKITUOIVIZWS2WDGTMNTU4Q,AF4QXCB32VC2DVE703...	NaN	

[5 rows x 1350 columns]

Matrix Sparsity: 99.92%

User-User Collaborative Filtering (cosine similarity)

```
[71]: from sklearn.metrics.pairwise import cosine_similarity

# Fill NaN values with 0 to perform matrix operations
user_item_matrix_filled = user_item_matrix.fillna(0)

# Calculate User-User Similarity
user_sim = cosine_similarity(user_item_matrix_filled)
user_sim_df = pd.DataFrame(user_sim, index=user_item_matrix.index,
    ↪ columns=user_item_matrix.index)

print(f"User Similarity Matrix Shape: {user_sim_df.shape}")
# Comparison Note:
# Given the 99.92% sparsity, Matrix Factorization (SVD) would likely outperform
# simple Cosine Similarity by uncovering latent patterns rather than relying
# on direct overlaps between sparse user vectors.
```

User Similarity Matrix Shape: (1193, 1193)

```
[59]: def get_user_user_recommendations(user_id, num_recs=5):
    if user_id not in user_item_matrix.index:
        return "User ID not found in the matrix."

    # Get similarity scores for the target user
    similar_users = user_sim_df[user_id].sort_values(ascending=False)[1:11]

    # Get products highly rated by similar users but not yet rated by the
    ↪target user
    user Rated_products = user_item_matrix.loc[user_id].dropna().index

    similar_users_ratings = user_item_matrix.loc[similar_users.index].mean().
    ↪sort_values(ascending=False)

    recs = similar_users_ratings.drop(labels=user Rated_products).head(num_recs)

    return
    ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'].
    ↪isin(recs.index)][['product_name', 'category_level_1', 'rating']]

# Test with a sample user from the matrix
sample_user = user_item_matrix.index[0]
print(f"Recommendations for user: {sample_user}")
display(get_user_user_recommendations(sample_user))
```

Recommendations for user: AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAHJK6NGYH
FMA,AGYURQ3476BNT4D2046THXEUY3SA,AFPMB5BIEX450Q6UCQWPDG55GWLQ,AGWJU3WUQBDQYPSYAJ
SR3AKBLCOA,AEOVUNFCIFV2230536GVW5JHZKOA

	product_name	category_level_1 \
348	Samsung Galaxy M04 Dark Blue, 4GB RAM, 64GB St...	Electronics
354	Samsung Galaxy M04 Light Green, 4GB RAM, 64GB ...	Electronics
454	Samsung Galaxy M04 Dark Blue, 4GB RAM, 128GB S...	Electronics
995	PHILIPS Digital Air Fryer HD9252/90 with Touch...	Home&Kitchen
1108	Kitchenwell Multipurpose Portable Electronic D...	Home&Kitchen

	rating
348	4.2
354	4.2
454	4.2
995	4.4
1108	4.3

Item-Item Collaborative Filtering (cosine or Pearson) Unlike user-user filtering, this approach looks for products that are similar based on how users have rated them.

```
[60]: from sklearn.metrics.pairwise import cosine_similarity

# Transpose the user-item matrix to get item-user matrix
item_user_matrix = user_item_matrix.T.fillna(0)

# Calculate Item-Item Similarity
item_sim = cosine_similarity(item_user_matrix)
item_sim_df = pd.DataFrame(item_sim, index=item_user_matrix.index,
    ↪ columns=item_user_matrix.index)

print(f"Item Similarity Matrix Shape: {item_sim_df.shape}")
display(item_sim_df.head())
```

Item Similarity Matrix Shape: (1350, 1350)

product_id	B002PD61Y4	B002SZEOLG	B003B00484	B003L62T7W	B004IO5BMQ	\
product_id						
B002PD61Y4	1.0	0.0	0.0	0.0	0.0	
B002SZEOLG	0.0	1.0	0.0	0.0	0.0	
B003B00484	0.0	0.0	1.0	0.0	0.0	
B003L62T7W	0.0	0.0	0.0	1.0	0.0	
B004IO5BMQ	0.0	0.0	0.0	0.0	1.0	

product_id	B005FYNT3G	B005LJQMCK	B005LJQMZC	B006LWOWDQ	B0073QGKAS	...	\
product_id							
B002PD61Y4	0.0	0.0	0.0	0.0	0.0	...	
B002SZEOLG	0.0	0.0	0.0	0.0	0.0	...	
B003B00484	0.0	0.0	0.0	0.0	0.0	...	
B003L62T7W	0.0	0.0	0.0	0.0	0.0	...	
B004IO5BMQ	0.0	0.0	0.0	0.0	0.0	...	

product_id	B0BP18W8TM	B0BP7XLX48	B0BP89YBC1	B0BPBG712X	B0BPBXNQQT	\
product_id						
B002PD61Y4	0.0	0.0	0.0	0.0	0.0	
B002SZEOLG	0.0	0.0	0.0	0.0	0.0	
B003B00484	0.0	0.0	0.0	0.0	0.0	
B003L62T7W	0.0	0.0	0.0	0.0	0.0	
B004IO5BMQ	0.0	0.0	0.0	0.0	0.0	

product_id	B0BPCJM7TB	B0BPJBTB3F	B0BQ3K23Y1	B0BQRJ3C47	B0BR4F878Q
product_id					
B002PD61Y4	0.0	0.0	0.0	0.0	0.0
B002SZEOLG	0.0	0.0	0.0	0.0	0.0
B003B00484	0.0	0.0	0.0	0.0	0.0
B003L62T7W	0.0	0.0	0.0	0.0	0.0
B004IO5BMQ	0.0	0.0	0.0	0.0	0.0

[5 rows x 1350 columns]


```
[61]: def get_item_item_recommendations(product_id, num_recs=5):
    if product_id not in item_sim_df.index:
        return "Product ID not found in the matrix."

    # Get similarity scores for the target product
    similar_items = item_sim_df[product_id].sort_values(ascending=False)[1:
    ↪num_recs+1]

    # Get metadata for the recommended products
    recs = []
    ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'].
    ↪isin(similar_items.index)]

    return recs[['product_name', 'category_level_1', 'rating', 'actual_price']]

# Test with a sample product from the matrix
sample_product = item_sim_df.index[0]
sample_product_name = []
    ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'].
    ↪isin([sample_product])]['product_name']
sample_product_name.columns = ['']
print('Product id =>', sample_product)
print('Product name =>', sample_product_name.values[0])
print('\n')
print("Recommendations for this product")
display(get_item_item_recommendations(sample_product))
```

Product id => B002PD61Y4

Product name => D-Link DWA-131 300 Mbps Wireless Nano USB Adapter (Black)

Recommendations for this product

	product_name \	category_level_1	rating	actual_price
238	Hi-Mobiler iPhone Charger Lightning Cable,2 Pa...	Computers&Accessories	4.0	799.0
630	HP 150 Wireless USB Mouse with Ergonomic and a...	Computers&Accessories	4.0	899.0
1058	Eureka Forbes Supervac 1600 Watts Powerful Suc...	Home&Kitchen	4.2	9999.0
1221	AGARO Classic Portable Yogurt Maker, 1.2L Capa...	Home&Kitchen	4.1	1295.0
1305	Glen 3 in 1 Electric Multi Cooker - Steam, Coo...	Home&Kitchen	4.1	2495.0

Item Similarity Matrix: The user-item matrix is transposed to calculate similarities between

products. This allows the system to recommend items that ‘people who liked this also liked.’

Functionality: The `get_item_item_recommendations` function identifies the top 5 most similar products based on rating overlaps.

Test Results: For the sample product (a Computer Accessory), the system suggested related tech items like a wireless mouse and charging cables, as well as highly-rated household appliances like a vacuum cleaner and multi-cooker, reflecting broader purchasing patterns in the dataset.

Comparison of User-Based and Item-Based Collaborative Filtering for this dataset:

1. User-User Collaborative Filtering

- **How it works:** Finds users with similar rating histories and suggests items they liked which the target user hasn’t seen.
- **Observations:** In this dataset, user-user filtering produced recommendations across different categories (e.g., suggesting an Air Fryer to a user who bought a phone). This is great for discovery and serendipity.
- **Challenges:** Because our matrix is 99.92% sparse, it is very hard to find users who have rated the exact same products, often leading to less reliable similarity scores.

2. Item-Item Collaborative Filtering

- **How it works:** Finds products that are commonly rated together by the same users.
- **Observations:** This produced highly relevant technical matches (e.g., suggesting a mouse and cables for a USB adapter). It is generally more stable because product relationships don’t change as quickly as user preferences.
- **Efficiency:** Usually preferred for large e-commerce sites like Amazon because there are often more ratings per item than per user, making the similarity scores more robust.

Summary

For this specific case, Item-Item filtering appeared to give more logically consistent results, whereas User-User filtering provided broader, more exploratory suggestions.

Section E: Hybrid Recommender (Content + Collaborative):

Design a hybrid strategy: $\text{Score fusion: } 0.6 * \text{CF_score} + 0.4 * \text{Content_score}$

Let’s select a sample product ID, extract its similarity scores from both the content-based (`combined_vectors_cosine_sim`) and collaborative (`item_sim_df`) matrices, and then align and normalize them to ensure they are on the same scale for the hybrid recommender.

```
[62]: from sklearn.preprocessing import MinMaxScaler

# 1. Select a sample product ID
sample_pid = 'B002PD61Y4'

# 2. Extract content-based similarity scores
# Find the integer index of the product in the unique products dataframe
```

```

idx = amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id']_
    ↳== sample_pid].index[0]
content_scores = combined_vectors_cosine_sim[idx]

# 3. Extract collaborative filtering similarity scores
# These are stored in item_sim_df where columns/index are product_ids
cf_scores_series = item_sim_df[sample_pid]

# 4. Align CF scores to the order of products in amazon_unique_product_ids_df
# This ensures the i-th element in both arrays refers to the same product
cf_scores_aligned = cf_scores_series.
    ↳reindex(amazon_unique_product_ids_df['product_id']).fillna(0).values

# 5. Normalize both sets of scores to [0, 1] range
scaler = MinMaxScaler()

# Reshape for scaler and transform
content_scores_norm = scaler.fit_transform(content_scores.reshape(-1, 1)).
    ↳flatten()
cf_scores_norm = scaler.fit_transform(cf_scores_aligned.reshape(-1, 1)).
    ↳flatten()

print(f'Similarity scores prepared for Product ID: {sample_pid}')
print(f'Content Scores - Min: {content_scores_norm.min():.4f}, Max: _
    ↳{content_scores_norm.max():.4f}')
print(f'CF Scores      - Min: {cf_scores_norm.min():.4f}, Max: {cf_scores_norm.
    ↳max():.4f}')
print(f'Aligned lengths: Content={len(content_scores_norm)},_
    ↳CF={len(cf_scores_norm)}')

```

```

Similarity scores prepared for Product ID: B002PD61Y4
Content Scores - Min: 0.0000, Max: 1.0000
CF Scores      - Min: 0.0000, Max: 1.0000
Aligned lengths: Content=1351, CF=1351

```

```

[63]: from sklearn.preprocessing import MinMaxScaler

def get_hybrid_item_recommendations(product_id, content_sim_matrix, cf_sim_df):
    # 1. Get the integer index of the input product_id
    if product_id not in amazon_unique_product_ids_df['product_id'].values:
        return "Product ID not found in unique products list."

    idx = _
    ↳amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'] ==_
    ↳product_id].index[0]

    # 2. Extract content similarity scores

```

```

content_scores = content_sim_matrix[idx]

# 3. Extract and align collaborative similarity scores
if product_id in cf_sim_df.index:
    cf_scores_series = cf_sim_df[product_id]
    cf_scores_aligned = cf_scores_series.
↪reindex(amazon_unique_product_ids_df['product_id']).fillna(0).values
else:
    cf_scores_aligned = np.zeros(len(amazon_unique_product_ids_df))

# 4. Normalize scores to [0, 1] range
scaler = MinMaxScaler()
content_norm = scaler.fit_transform(content_scores.reshape(-1, 1)).flatten()
cf_norm = scaler.fit_transform(cf_scores_aligned.reshape(-1, 1)).flatten()

# 5. Calculate Hybrid Score (40% Content, 60% Collaborative)
hybrid_scores = (0.4 * content_norm) + (0.6 * cf_norm)

# 6. Store scores in a copy of the dataframe
recommendations_df = amazon_unique_product_ids_df.copy()
recommendations_df['hybrid_score'] = hybrid_scores

return recommendations_df

print('Function get_hybrid_item_recommendations defined successfully.')

```

Function get_hybrid_item_recommendations defined successfully.

```

[64]: # 1. Define a sample product ID to test
sample_pid = 'B002PD61Y4'
sample_product_name = ↪
↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'].
↪isin([sample_pid])]['product_name']
sample_product_name.columns = ['']
sample_product_name = sample_product_name.values[0]

# 2. Call the hybrid recommendation function
hybrid_recs_df = get_hybrid_item_recommendations(sample_pid, ↪
↪combined_vectors_cosine_sim, item_sim_df)

# 3. Exclude the input product itself
filtered_recs = hybrid_recs_df[hybrid_recs_df['product_id'] != sample_pid]

# 4. Sort by hybrid_score in descending order
sorted_recs = filtered_recs.sort_values(by='hybrid_score', ascending=False)

# 5. Select the top 5 rows

```

```

top_5_hybrid = sorted_recs.head(5)

# 6. Display the recommendations with metadata
print('Top 5 Hybrid Recommendations for:\n')
print(f"Product id => {sample_pid}")
print(f"Product name => {sample_product_name}")
print('_' * 100)
display(top_5_hybrid[['product_name', 'category_level_1', 'actual_price', 'rating', 'hybrid_score']])

```

Top 5 Hybrid Recommendations for:

Product id => B002PD61Y4

Product name => D-Link DWA-131 300 Mbps Wireless Nano USB Adapter (Black)

```

-----
product_name      category_level_1 \
175 Skadioo WiFi Adapter for pc | Car Accessories,... Computers&Accessories
788 D-Link DIR-615 Wi-fi Ethernet-N300 Single_band... Computers&Accessories
90  Gizga Essentials USB WiFi Adapter for PC, 150 ... Computers&Accessories
303 NK STAR 950 Mbps USB WiFi Adapter Wireless Net... Computers&Accessories
43  TP-Link Nano AC600 USB Wi-Fi Adapter(Archer T2... Computers&Accessories

actual_price  rating  hybrid_score
175          499.0    3.7      0.397204
788          1800.0    4.1      0.397008
90           800.0    3.6      0.396680
303           999.0    4.2      0.396638
43          1599.0    4.3      0.396532

```

0.4.1 Hybrid Strategy Summary

How does the hybrid approach balance semantic product relevance with actual user purchasing behavior patterns? The hybrid approach balances these two signals by applying a weighted average of their normalized scores. Semantic relevance is captured through **Content-Based Filtering (40% weight)**, which uses product embeddings and metadata to ensure items are functionally similar (e.g., matching a WiFi adapter with other networking tools). User purchasing patterns are captured through **Collaborative Filtering (60% weight)**, which prioritizes products that real customers frequently interact with or buy together. This 40/60 split ensures that while the system respects the physical characteristics of the product, it leans more heavily on actual market behavior and community preferences.

0.4.2 Data Analysis Key Findings

- **Successful Data Alignment:** Similarity scores from two distinct models were successfully aligned for **1,351 products**, ensuring that the i-th score in the content matrix refers to the same product as the i-th score in the collaborative matrix.

- **Normalization Strategy:** Using `MinMaxScaler`, both content and collaborative scores were scaled to a strict **[0.0, 1.0] range**. This step was critical to prevent one model’s score magnitude from dominating the other before the weighting was applied.
- **Weighted Scoring:** The implementation used a ratio of **40% content-based similarity** and **60% collaborative similarity**, favoring historical user interaction data while maintaining category relevance.
- **Top Recommendations Performance:** For a sample product (Wi-Fi Adapter B002PD61Y4), the system generated top 5 recommendations including brands like Skadiioo, D-Link, and Gizga.
- **Score Stability:** The top hybrid scores were clustered around **0.39**, indicating a strong consensus between the two recommendation engines for the top-ranked items.

0.4.3 Insights or Next Steps

- **Mitigating Cold Start:** The inclusion of content-based filtering (40%) ensures that even if a product has limited purchase history, the system can still provide logical recommendations based on its category and description.
- **Weight Optimization:** Future iterations could involve A/B testing different weight ratios (e.g., 50/50 or 30/70) to see which combination leads to higher click-through rates or actual conversions in a production environment.

0.4.4 Comparison of Recommendation Quality: Hybrid vs. Individual Methods

Feature	Content-Based (Individual)	Collaborative (Individual)	Hybrid (40/60 Fusion)
Accuracy	High for technical specs/keywords.	High for matching user behavior.	Best of both worlds: Technical & Behavioral.
Cold Start	Handles new products perfectly.	Fails if products have no ratings.	Robust: Uses content if ratings are missing.
Sparsity	Not affected.	Heavily limited by 99.92% sparsity.	Mitigated: Fills gaps with semantic data.
Serendipity	Low (recommends ‘more of the same’).	High (finds hidden patterns).	Balanced: Logical but exploratory.

Key Conclusion: For this Amazon dataset, a **Hybrid approach** is superior because it prevents the ‘logical errors’ of pure collaborative filtering (which struggles with sparse data) and the ‘tunnel vision’ of pure content filtering (which ignores what users actually buy). By weighting Collaborative filtering at 60%, we prioritize market evidence while maintaining a 40% safety net of category and price relevance.

0.4.5 Evaluate hybrid system on:

A cold-start product (new product)

Identify a Cold-Start Product Find a product in `amazon_unique_product_ids_df` with zero or minimal rating counts to simulate a new item entry for testing the Cold Start problem.

```
[65]: # Find products with 0 rating_count to simulate a cold start
cold_start_candidates =
    ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['rating_count'] ==
    ↪0]

# Select the first product ID from the candidates
if not cold_start_candidates.empty:
    cold_start_pid = cold_start_candidates.iloc[0]['product_id']
    cold_start_product_name = cold_start_candidates.iloc[0]['product_name']
    cold_start_rating_count = cold_start_candidates.iloc[0]['rating_count']

    print(f"Cold-Start Product ID: {cold_start_pid}")
    print(f"Product Name: {cold_start_product_name}")
    print(f"Rating Count: {cold_start_rating_count}")
else:
    print("No products with 0 rating count found. Searching for products
    ↪missing from collaborative matrix instead.")
    # Fallback: Find a product ID in the unique DF that is not in the
    ↪collaborative matrix index
    missing_from_cf = [pid for pid in
    ↪amazon_unique_product_ids_df['product_id'] if pid not in item_sim_df.index]
    if missing_from_cf:
        cold_start_pid = missing_from_cf[0]
        row =
    ↪amazon_unique_product_ids_df[amazon_unique_product_ids_df['product_id'] ==
    ↪cold_start_pid].iloc[0]
        print(f"Cold-Start Product ID (Missing from CF): {cold_start_pid}")
        print(f"Product Name: {row['product_name']}")
        print(f"Rating Count: {row['rating_count']}")
```

Cold-Start Product ID: B0B94JPY2N

Product Name: Amazon Brand - Solimo 65W Fast Charging Braided Type C to C Data Cable | Suitable For All Supported Mobile Phones (1 Meter, Black)

Rating Count: 0

Run Hybrid Recommendations for Cold-Start Generate and analyze hybrid recommendations for the identified cold-start product ID.

```
[66]: import pandas as pd

# 1. Generate hybrid recommendations for the cold-start product
cold_start_recs_df = get_hybrid_item_recommendations(cold_start_pid,
    ↪combined_vectors_cosine_sim, item_sim_df)

# 2. Filter out the cold-start product itself
filtered_cold_start_recs = cold_start_recs_df[cold_start_recs_df['product_id'] !=
    ↪cold_start_pid]
```

```
# 3. Sort by hybrid_score and get the top 5
top_5_cold_start = filtered_cold_start_recs.sort_values(by='hybrid_score',
    ↪ascending=False).head(5)

# 4. Display results
print(f'Cold-Start Product: {cold_start_product_name}\n')
print('_' * 100)
print('Top 5 Hybrid Recommendations (Leveraging 40% Content Fallback):')
display(top_5_cold_start[['product_name', 'category_level_1', 'actual_price',
    ↪'rating', 'hybrid_score']])
```

Cold-Start Product: Amazon Brand - Solimo 65W Fast Charging Braided Type C to C Data Cable | Suitable For All Supported Mobile Phones (1 Meter, Black)

 Top 5 Hybrid Recommendations (Leveraging 40% Content Fallback):

	product_name	category_level_1	\
313	Amazon Brand - Solimo Fast Charging Braided Ty...	Computers&Accessories	
277	Amazon Brand - Solimo 3A Fast Charging Tough T...	Computers&Accessories	
37	Sounce 65W OnePlus Dash Warp Charge Cable, 6.5...	Computers&Accessories	
241	Wayona Usb Type C To Usb Nylon Braided Quick C...	Computers&Accessories	
161	Wayona Type C Cable Nylon Braided USB C QC 3.0...	Computers&Accessories	

	actual_price	rating	hybrid_score
313	599.0	4.1	0.398750
277	299.0	3.8	0.398041
37	999.0	4.0	0.397904
241	999.0	4.3	0.397776
161	999.0	4.3	0.397714

0.5 Summary:

How does the hybrid model effectively bridge the “Cold Start” problem? The hybrid model bridges the “Cold Start” problem by incorporating product metadata (categories, descriptions, and features) into the recommendation engine. When a new or unpopular product (like the Solimo 65W Data Cable) has zero ratings, the collaborative filtering component provides no data. However, the model uses a 40% content weight to calculate cosine similarity based on metadata, allowing it to generate relevant recommendations even in the total absence of user behavioral data.

0.5.1 Data Analysis Key Findings

- **Cold-Start Identification:** The product B0B94JPY2N (*Amazon Brand - Solimo 65W Fast Charging Braided Type C to C Data Cable*) was successfully identified as a cold-start candidate because it had a `rating_count` of exactly 0.
- **Collaborative Filtering Failure:** Because the item had no interaction history, its collaborative filtering score was 0.0, meaning a purely behavior-based system would have failed to

provide any related items.

- **Metadata Fallback Performance:** By leveraging the 40% content-based weighting, the system generated top-tier recommendations with hybrid scores between **0.397** and **0.399**.
- **Semantic Relevance:** All top 5 recommendations were highly relevant products (e.g., *Sounce 65W OnePlus Dash Warp Charge Cable* and other *Solimo Type C cables*), proving that textual and categorical similarity is a reliable proxy for user intent when ratings are missing.

0.5.2 Insights or Next Steps

- **Weight Optimization:** Since the content-based component effectively identified highly relevant replacements for a product with zero data, future iterations could test dynamic weighting—automatically increasing the content weight to 100% when a product’s rating count is below a certain threshold.
- **Enrich Metadata:** To further improve cold-start performance, additional metadata features like technical specifications (voltage, wattage, material) could be vectorized to differentiate between very similar electronic accessories.

0.5.3 Evaluate hybrid system on:

A cold-start user (few reviews)

Identify a cold-start user by searching the `reviews_per_user` data for a user with the minimum number of reviews (e.g., 1 review). Extract the `product_id` of the specific item this user has rated to serve as the seed for recommendations. Use the `get_hybrid_item_recommendations` function with this seed ID to generate the top 5 hybrid suggestions, demonstrating how the system leverages item similarity and metadata when the user’s specific network is sparse. Finally, display and evaluate the relevance of these recommendations to show how the hybrid approach effectively addresses the user cold-start scenario by falling back on item-to-item relationships and content features.

Identify a Cold-Start User I will identify a cold-start user by filtering the existing `reviews_per_user` dataframe for users with only one review and selecting the first one.

```
[67]: # Filter reviews_per_user for users with exactly 1 review
cold_start_user_list = reviews_per_user[reviews_per_user['Total Reviews'] == 1]

# Select the first user_id from the filtered list
if not cold_start_user_list.empty:
    cold_start_user_id = cold_start_user_list.iloc[0]['user_id']
    print(f"Selected Cold-Start User ID: {cold_start_user_id}")
    print(f"Total Reviews for this user: {cold_start_user_list.iloc[0]['Total_
↵Reviews']}")
else:
    print("No users with only 1 review found.")
```

Selected Cold-Start User ID: AGPVN62QTZNEHCVDPA4237YQ5VMQ,AF7KNSLEDN6UCGACICGMVX
RW6FZQ,AERIRWR6GSTILA6LAAFDAG3FENNA,AG3SI2KQY3UNEQJOKH565UD4Y6FA,AGBDMC7RQ25U6AE
K5YA5TLP4S5HQ,AGS23EI6FG7FXH3XR4HGYLJF5UAA,AGLDR4PNZ6ZWNGFV5EEV7BL6LAAA,AGE5JVVF

COPUTGK7F2PUUST4OWOA

Total Reviews for this user: 1

Now that I have identified a cold-start user, I need to find the product they interacted with to use as a seed for generating recommendations.

```
[68]: # Find the product_id for the review made by the cold-start user
seed_product_id = amazon[amazon['user_id'] == cold_start_user_id]['product_id'].
    ↪iloc[0]
seed_product_info = amazon[amazon['product_id'] ==
    ↪seed_product_id][['product_name', 'category_level_1']].drop_duplicates()

print(f"Seed Product ID for User: {seed_product_id}")
print(f"Product Name: {seed_product_info['product_name'].values[0]}")
print(f"Category: {seed_product_info['category_level_1'].values[0]}")
```

Seed Product ID for User: B08444S68L

Product Name: OPPO A31 (Mystery Black, 6GB RAM, 128GB Storage) with No Cost EMI/Additional Exchange Offers

Category: Electronics

With the seed product identified for the cold-start user, I will now use the previously defined hybrid recommendation function to generate suggestions based on that seed product's metadata and limited interaction history.

```
[69]: # Generate hybrid recommendations for the cold-start user's seed product
hybrid_user_recs_df = get_hybrid_item_recommendations(seed_product_id,
    ↪combined_vectors_cosine_sim, item_sim_df)

# Filter out the seed product itself
filtered_user_recs = hybrid_user_recs_df[hybrid_user_recs_df['product_id'] !=
    ↪seed_product_id]

# Sort and get top 5
top_5_user_cold_start = filtered_user_recs.sort_values(by='hybrid_score',
    ↪ascending=False).head(5)

print(f'Hybrid Recommendations for Cold-Start User (Seed Product:
    ↪{seed_product_info["product_name"].values[0]})\n')
print('_' * 100)
display(top_5_user_cold_start[['product_name', 'category_level_1',
    ↪'actual_price', 'rating', 'hybrid_score']])
```

Hybrid Recommendations for Cold-Start User (Seed Product: OPPO A31 (Mystery Black, 6GB RAM, 128GB Storage) with No Cost EMI/Additional Exchange Offers)

product_name category_level_1 \

383	OPPO A74 5G (Fantastic Purple,6GB RAM,128GB St...	Electronics
435	OPPO A74 5G (Fluid Black, 6GB RAM, 128GB Stora...	Electronics
491	Redmi Note 11T 5G (Aquamarine Blue, 6GB RAM, 1...	Electronics
481	Redmi Note 11T 5G (Matte Black, 6GB RAM, 128GB...	Electronics
510	Redmi Note 11T 5G (Stardust White, 6GB RAM, 12...	Electronics

	actual_price	rating	hybrid_score
383	20990.0	4.2	0.398773
435	20990.0	4.2	0.398684
491	20999.0	4.1	0.397583
481	20999.0	4.1	0.397556
510	20999.0	4.1	0.397528

0.6 Summary:

How does the hybrid approach address user cold-start scenarios? The hybrid approach addresses cold-start scenarios by pivoting from a user-centric model to an item-centric model. When a user has a sparse profile (e.g., only one review), the system uses that single data point as a “seed.” It then leverages item-to-item relationships—combining collaborative filtering patterns with content-based metadata—to find similar products. This ensures the user receives relevant suggestions based on the specific attributes and community associations of the item they interacted with, even without a deep personal history.

0.6.1 Data Analysis Key Findings

- **Cold-Start Identification:** A user was successfully identified with the minimum possible interaction history: exactly **1 review**.
- **Seed Product Context:** The cold-start user’s singular interest was an “**OPPO A31 (Mystery Black, 6GB RAM, 128GB Storage)**” within the **Electronics** category.
- **Recommendation Relevance:** The system generated top 5 recommendations that were highly contextually relevant, including the **OPPO A74 5G** and **Redmi Note 11T 5G**.
- **Scoring Consistency:** The top recommendations achieved narrow and consistent hybrid scores ranging from approximately **0.397 to 0.399**, indicating a strong consensus between the content features and item-similarity metrics.
- **Attribute Matching:** The fallback mechanism successfully matched technical specifications (such as 6GB RAM and 128GB Storage) and category levels, proving the effectiveness of metadata integration.

0.6.2 Insights or Next Steps

- **Incorporate Diversity Metrics:** While the recommendations are highly relevant, they are very similar to the seed product. Future iterations could benefit from a “diversity boost” to introduce related categories (e.g., phone cases or chargers) alongside similar smartphone models.
- **Weight Tuning:** Analyze whether the hybrid score should lean more heavily on content metadata versus collaborative filtering as the number of user reviews increases, creating a dynamic weighting system based on profile depth.

Suggest how to improve hybrid performance further using real-world constraints like:

- Popularity
- Recent purchases
- Product availability

Enhancing Hybrid Recommendations with Real-World Constraints To optimize the recommendation engine for a production environment, we can incorporate the following three layers into our scoring logic:

1. Popularity Bias (Bayesian Averaging) * **The Problem:** High similarity scores might occasionally surface ‘niche’ or low-rated products. * **The Fix:** Add a **Popularity Multiplier**. Instead of using raw ratings, use a Bayesian Average that accounts for the `rating_count`. This ensures that a 5.0-star product with 2 reviews doesn’t outrank a 4.8-star product with 20,000 reviews.

2. Recency & Temporal Decay * **The Problem:** Collaborative filtering might suggest products based on old trends (e.g., last year’s smartphone model). * **The Fix:** Implement a **Time-Decay Function**. Multiply the similarity score by a factor like $e^{-\lambda t}$, where t is the time since the product’s last review or purchase. This keeps the recommendations ‘fresh’ and aligned with current market trends.

3. Availability & Business Constraints * **The Problem:** Recommending out-of-stock items leads to a poor user experience and lost conversion. * **The Fix:** Implement a **Hard Constraint Filter**. Before the final sort, join the recommendation list with an inventory status table. If `is_in_stock == False`, the product is either removed or moved to a ‘Pre-order’ section with a penalty to its hybrid score.

Revised Hybrid Score Formula Example:

$$Final\ Score = (w_1 \cdot Content + w_2 \cdot Collaborative) \times Recency\ Factor \times Popularity\ Weight$$

0.7 Section F: Bonus: Business Strategy & Deployment

Which model works best for new users?

0.7.1 Recommendation Strategy for New Users (Cold-Start)

Winner: Hybrid Content-Collaborative Model For new users who have just joined the platform and have zero or very few reviews, **Collaborative Filtering** is ineffective because there is no historical data to calculate similarities with other users.

Why the Hybrid Model is Best: 1. **Immediate Relevance:** As soon as a new user clicks on a single product, the Hybrid model uses the **Content-Based (40%)** component to find items with similar technical specs, categories, and price points. 2. **Market Guidance:** It still applies the **Collaborative (60%)** logic to the *seed item* itself. This means the user sees products that are not just similar, but are also confirmed ‘best-sellers’ among the wider community. 3. **Path to Personalization:** As the user continues to interact, the system dynamically shifts from ‘Item-Similarity’ (Content) to ‘User-Preference’ (Collaborative) as their profile grows.

Business Impact: This prevents the ‘Empty State’ problem, increasing the likelihood of the first purchase and higher user retention.

Which model works best for returning users?

0.7.2 Recommendation Strategy for Returning Users

Winner: Item-Item Collaborative Filtering For users with an existing history of interactions, **Item-Item Collaborative Filtering** (which we implemented using Cosine Similarity) is the most effective tool currently in our pipeline.

Why Item-Item Filtering is Best for Returning Users:

1. **Stable Product Relationships:** Unlike user preferences which can be flighty, the relationship between products (e.g., people who buy a specific DSLR often buy a specific tripod) is statistically stable and highly reliable for cross-selling.
2. **Overcoming Sparsity:** While User-User filtering struggles with our 99.92% sparsity, items generally have more ratings than individual users, making the item-similarity scores more robust and less likely to result in ‘empty’ recommendations.
3. **Real-World Context:** It allows for the ‘Frequently Bought Together’ functionality, which is a major driver of Average Order Value (AOV) on platforms like Amazon.

Business Impact: High-quality item-to-item recommendations for returning users directly improve **Customer Lifetime Value (CLV)** by surfacing relevant accessories and upgrades based on their previous purchases.

How can we recommend products with no ratings?

0.7.3 How to Recommend Products with No Ratings

Strategy: Content-Based Fallback Products with no ratings suffer from the **Item Cold-Start** problem. Since no user-item interactions exist, similarity cannot be calculated via Collaborative Filtering.

The Solution:

1. **Semantic Mapping:** Use the **Content-Based Model** (TF-IDF or Embeddings) to convert the new product’s metadata (name, description, category) into a vector.
2. **Attribute Matching:** Calculate the Cosine Similarity between this new vector and all existing products in the catalog.
3. **Cross-Promotion:** Surface the new product on the pages of highly similar, established products under a “Similar Items” or “New Arrivals You Might Like” section.
4. **Initial Discovery:** Use the user’s past interest in specific categories or keywords to push the new product to their feed, effectively using the content features as a proxy for potential interest.

Business Impact: This ensures that new inventory gets immediate visibility, allowing it to collect its first ratings and eventually transition into the more powerful Collaborative Filtering models.

How would you deploy this system in production? Mention tools/technologies.

0.7.4 Production Deployment Architecture

To move this recommendation system into production, we would use a **Lambda Architecture** (Batch + Real-time layers).

1. Data Storage & Feature Store

- **AWS S3 / Google Cloud Storage:** For raw data lake storage.
- **Feature Store (e.g., Feast or AWS SageMaker Feature Store):** To store and serve normalized user and item features (like our combined vectors) with low latency.

2. Retrieval Layer (Candidate Generation)

- **Vector Database (e.g., Pinecone, Milvus, or Weaviate):** To store the high-dimensional embeddings. We use **Approximate Nearest Neighbor (ANN)** search to retrieve the top 100 potential products in milliseconds.
- **Redis:** For caching frequently accessed user profiles and ‘Frequently Bought Together’ pre-computed lists.

3. Scoring & Ranking Layer

- **Inference Service (e.g., AWS SageMaker or TFX):** A microservice that takes the candidates from the retrieval layer and applies our **Hybrid Scoring Logic** ($0.6 \text{ CF} + 0.4 \text{ Content}$) to rank them.
- **A/B Testing (e.g., Optimizely):** To test different weight ratios (e.g., 50/50 vs 40/60) on live traffic.

4. API & Orchestration

- **FastAPI / Flask:** To wrap the model in a REST API that the frontend (website/app) can call.
- **Apache Airflow:** To schedule daily re-training jobs and update the similarity matrices as new ratings come in.

5. Monitoring

- **Prometheus & Grafana:** To monitor API latency and system health.
- **Evidently AI:** To track **Model Drift** (ensuring the model stays accurate as product trends change).

What KPIs should Amazon track to measure success?

0.7.5 KPIs to Measure Recommendation Success

To evaluate the performance of the hybrid recommendation system, the following Key Performance Indicators (KPIs) should be tracked:

1. Business Metrics (User Impact)

- **Click-Through Rate (CTR):** The percentage of users who click on a recommended product. This measures the immediate relevance of the suggestions.
- **Conversion Rate (CVR):** The percentage of clicks on recommendations that lead to an actual purchase. This is the ultimate measure of business value.
- **Average Order Value (AOV):** By suggesting relevant accessories (cross-selling), a good system should increase the total amount spent per transaction.
- **Customer Lifetime Value (CLV):** Tracking whether personalized recommendations lead to higher retention and repeat purchases over time.

2. Technical Metrics (Model Accuracy)

- **Root Mean Square Error (RMSE):** Measures the difference between predicted ratings and actual ratings (primarily for collaborative filtering/SVD models).
- **Precision @ K:** Of the top 'K' recommendations shown to a user, how many were actually relevant or interacted with?
- **Recall @ K:** How many of the items the user eventually bought were actually captured in the top 'K' recommendations?

3. Catalog & Diversity Metrics

- **Catalog Coverage:** The percentage of the total product catalog that is actually being recommended. A system with low coverage only recommends 'blockbusters,' missing niche sales.
- **Serendipity:** A measure of how 'surprising' or 'novel' the recommendations are, ensuring the user isn't just seeing the same brands repeatedly.
- **Churn Rate of Recommendations:** How often the recommendation list refreshes to keep the user experience feeling new.